PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

REAL-TIME ML BASED PHISHING DETECTION AND PREVENTION SYSTEM

A PROJECT REPORT

Submitted by

K SANJAY- 20221CCS0062

PUNITH C- 20221CCS0066

OMKAR- 20221CCS0070

Under the guidance of,

Ms. SOUMYA

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

BONAFIDE CERTIFICATE

Certified that this report "REAL-TIME ML BASED PHISHING DETECTION AND PREVENTION SYSTEM" is a bonafide work of "K SANJAY(20221CCS0062),PUNITH C (20221CCS0066), OMKAR(20221CCS0070)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING, CYBER SECURITY during 2025-26.

21/04/2026

Ms. Soumya
Assistant Professor
Project Guide
PSCS
Presidency University

Dr. Geetha Arjunan
Associate Professor
Program Project Coordinator
PSCS
Presidency University

Dr. Sampath A K
Professor
School Project Coordinators
PSCS
Presidency University

Dr. Anandaraj S.P
Professor & Head of the Department
PSCS
Presidency University

Dr. Shakkeera L
Associate Dean
PSCS
Presidency University

Sl. No	Name of the Examiner	Signature	Date
1.	Ameen Khanum		21/4/26
2.	Mr. LAKSHMISHA S.K		21/4/26

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named K SANJAY, PUNITH C, OMKAR, hereby declare that the project work titled "REAL-TIME ML BASED PHISHING DETECTION AND PREVENTION SYSTEM" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY) during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

NAME	ROLL NUMBER	
K SANJAY	20221CCS0062	
PUNITH C	20221CCS0066	
OMKAR	20221CCS0070	

PLACE: BENGALURU

DATE 21-04-2026.

ACKNOWLEDGEMENT

For completing this project work, we have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Ms. Soumya, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Anandaraj S P**, Professor, Head of the Department, Presidency School of Computer Science and Engineering Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Shakkeera L**, Associate Dean, Presidency School of computer Science and Engineering (CYBER SECURITY) and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, PSCS** Project Coordinator**, Dr. Geetha Arjunan, Program Project Coordinator**, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

K SANJAY

PUNITH C

OMKAR

ABSTRACT

With the rise in services as well as digital platforms, there is also an increase in the number of phishing attacks. Such a situation is dangerous as it involves identity theft, financial losses as well as data leakage caused by such threats. To address the above-stated problem we have created a software called PhishGuard. It is a machine learning program which allows for preventing phishing attacks. PhishGuard is a computer program which is designed to be used on an Android device and which is aimed at protection from malicious actions on web pages as well as URLs in particular. PhishGuard can be used in two different ways. Firstly, a link should be entered in order to receive information about its nature and whether it is legitimate or not. Secondly, a constant functioning of PhishGuard takes place. So, if the user enters the website on the PhishGuard software the application determines whether the link is legitimate or not. Otherwise, when the user's PhishGuard functions all the time, all the visited URLs on his device are being checked as a measure of safety against phishing attacks.PhishGuard analyzes URLs before visiting a web page. the process starts with a domain filtering step that helps identify safe websites and exclude them from being considered in order to improve performance. The analysis is performed separately for other domains, during which feature extraction is carried out by studying various attributes of a website, such as URL length, presence of special characters, usage of IP addresses, and domain-based features. Further, the obtained data goes through a trained Neural Network model that has been transformed into a TensorFlow Lite version to ensure efficient inference on-device.

In the course of developing the model, different machine learning techniques, such as Neural Networks and XGBoost, have been trained and evaluated based on available datasets in order to select the best algorithm for phishing detection. Even though both models performed quite efficiently, it was decided to utilize a Neural Network algorithm due to its better efficiency and performance. This is achieved through the employment of a monitoring technique that employs a virtual private network (VPN). The monitoring technique allows the system to examine the traffic from the device and determine any abnormal activity happening in the application. Consequently, the efficiency of detection is enhanced through the examination of not only manually but also automatically via real-time actions in the browser.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	Declaration	i
	Acknowledgement	ii
	Abstract	iii
	Table Of Contents	iv-v
	List Of Figures	vi
	List Of Tables	vii
	Abbreviations	viii-ix
1.	Introduction	1-6
	1.1 Background and Motivation	1
	1.2 Statistics of Project	1-2
	1.3 Prior Existing Technologies	2
	1.4 Proposed Approach	2-3
	1.5 Objectives	4
	1.6 SDGs	4-5
	1.7 Overview of Project Report	5-6
2.	Literature Review	7-8
3.	Methodology	9-19
4.	Project Management	20-23
	4.1 Project Timeline	20-21
	4.2 Risk Analysis	21-22
	4.3 Project Budget	23
5.	Analysis And Design	24-36
	5.1 Requirements	24-26
	5.2 Block Diagram	26
	5.3 System Flow Chart	26-27
	5.4 Hardware Components and Specification	27-28
	5.5 Software Components and Specification	28
	5.6 Functional Modules of Phishguard	28-29

	5.7 Datasets and Data Resources	29
	5.8 Functional Modules and Brief Features	29-30
	5.9 Selection of Devices and Software	30-31
	5.10 Designing Units	31-33
	5.11 Design Models	33-34
	5.12 Iot Deployment Model	34-35
	5.13 Functional View	35
	5.14 Operational View	36
6.	Hardware, Software and Simulation	37-44
	6.1 Hardware	37
	6.2 Software Development Tools	37-39
	6.3Software Code	39-42
	6.4 **S**imulation	42-44
7.	Results And Evaluations	45-51
	7.1 Test Points	45
	7.2 Test Plan	45-47
	7.3 Test Result	47-48
	7.4 Key Insights	48-49
	7.5 Model Training Results	49-50
	7.6 Admin Portal Results	50-51
8.	Social, Legal, Ethical, Sustainability and Safety Aspects	52-53
	8.1 Social Aspects	52
	8.2 Legal Aspects	52
	8.3 Ethical Aspects	52
	8.4 Sustainability Aspects	52
	8.5 Safety Aspects	52-53
9.	Conclusion	54-55
	References	56
	Base Paper	56
	Appendix	57-60

LIST OF FIGURES

Sl. No.	Figure Name	Figure Caption	Page No.
1	Figure 1.6	Sustainable development goals	5
2	Figure 3.1	V-Model Methodology	9
3	Figure 3.3	Block diagram of the Phishguard architecture	13
4	Figure 3.4	Unit Design Flowchart	15
5	Figure 4.1	Timeline Gantt Chart	21
6	Figure 5.2	Functional block diagram	26
7	Figure 5.3	System flow chart of phishguard	27
8	Figure 5.10	Unit Diagram of Phishguard	32
9	Figure 5.12	Iot deployment model of phishguard	34
10	Figure 5.13	functional view of phishguard	35
11	Figure 5.14	operational view of Phishguard	36
12	Figure 7.1	PhishGuard Interface	45
13	Figure 7.2.1	test results of URL Scanner	46
14	Figure 7.2.2	PhishGuard operational modes	47
15	Figure 7.3.1	test results of auto protection	48
16	Figure 7.3.2	test results of auto protection	48
17	Figure 7.5	Results of Model trained	50
18	Figure 7.6	Admin portal	51

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 2.1	Summary of Literature reviews	8
2	Table 4.1.1	Project planning timeline	20
3	Table 4.1.2	Project implementation timeline	21
4	Table 4.2	Project phase risk matrix	22-23
5	Table 5.4	Hardware Components and Specifications	27-28
6	Table 5.5	Software Components and Requirements	28
7	Table 5.6	Functional Modules of the Phishguard System	28-29
8	Table 5.7	Datasets and Data Resources	29
9	Table 5.8	Functional Modules and Brief Features	29-30
10	Table 5.9	Comparison of Selected Devices and Software	31

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
URL	Uniform Resource Locator
API	Application Programming Interface
REST	Representational State Transfer
VPN	Virtual Private Network
NN	Neural Network
TFLITE	TensorFlow Lite
XGBOOST	Extreme Gradient Boosting
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
DNA	Domain Name System
IP	Internet Protocol
UI	User Interface
UX	User Experience
APK	Android Package Kit
SDK	Software Development Kit
JSON	Integrated Development Environment
IoT	JavaScript Object Notation
CSV	Comma-Separated Values
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
TP	True Positive
TN	True Negative
FP	False Positive

FN	False Negative
CPU	Central Processing Unit
RAM	Random Access Memory

Chapter 1

Introduction

Phishing has emerged as a significant cybersecurity hazard, compromising on misleading URLs, whilst stealing sensitive user data. This project will focus on producing a smart mobile-based detection system that can properly detect malicious links in real time. As cyberattacks grow steadily, current blacklist and rule-of-thumb detection methods are no longer effective to identify threats. This therefore satisfies the requirement of a strong solution based on machine learning methods, enhancing the accuracy of detection and user safety compared to the conventional ones, as well as providing effective, on-device protection.

1.1 Background and motivation

The quick advancement in communication systems in the digital world has turned the smartphone into an inseparable part of our lives. Nowadays, people use their smartphones not only for communication purposes but also for financial transactions, social networking, shopping, and working. On the other hand, due to increasing dependence on mobile communication systems, cyber attackers can use various channels to launch cyberattacks against their victims. Phishing attacks have proven themselves to be one of the most popular types of cybercrime.

Generally speaking, phishing attacks are designed by hackers to manipulate their victims' minds by getting access to confidential information from them. Attackers utilize different methods to steal valuable information, like sending spam emails, SMS phishing attacks, and using malicious links. With the passage of time, phishing attacks have become much more complex than ever before.Recent trends show that phishing attacks are responsible for a major share of cybersecurity incidents globally. Mobile platforms, in particular, are highly vulnerable due to frequent internet usage and user interaction with links. Traditional detection techniques such as blacklists and rule-based systems are no longer effective in identifying new and dynamic phishing attacks, especially zero-day threats.In order to overcome these obstacles, recent techniques in the detection of phishing attacks have moved towards adopting machine learning-based approaches that can detect patterns and outliers from phishing attempts. In this project, we will present PhishGuard, a cutting-edge mobile-based solution for detecting phishing that utilizes real-time monitoring of URLs, features, and machine learning.

1.2 Statistics of Project

Phishing attacks have witnessed a significant rise in frequency and sophistication levels. Research suggests that phishing is responsible for around 80–85% of cyber security events, thus being among the primary methods employed against organizations.More than 3.4 billion phishing emails are sent

each day, and almost 90% of all data breaches can be attributed to phishing attacks. The number of phishing attacks has risen by 36% in the first six months of 2025 alone, signaling the swift increase in cyber security incidents.

Types of phishing attacks vary widely. There are several different kinds of phishing attacks:

- Credential harvesting attacks represent approximately 45% of phishing events
- Financial fraud attacks comprise roughly 28% of phishing events
- Malware attacks make up close to 20% of phishing attacks
- Advanced phishing techniques have been developed in modern times. About 75% of phishing attacks employ polymorphism to change URLs. Typosquatting, domain spoofing, and homograph attacks are some of the most common advanced techniques used in phishing campaigns.
- The mobile phone has become a critical element in the effectiveness of phishing attacks, with around 65% of phishing attempts being conducted through mobile platforms. The immediate requirement for mobile phishing detection tools such as PhishGuard is therefore essential.

1.3 Prior Existing Technologies

Before advanced systems like PhishGuard were developed, phishing detection mainly relied on traditional methods such as blacklist-based services like Google Safe Browsing and PhishTank. These systems worked by comparing URLs against a database of known phishing websites, which meant they were only effective for previously identified threats and failed to detect new or evolving phishing attacks. Web browsers such as Google Chrome and Mozilla Firefox also provided built-in phishing protection, but their functionality was limited only to browser activity and did not extend to other applications like messaging or third-party apps.Additionally, some systems focused on email and SMS filtering, commonly seen in applications like Gmail, where suspicious links could be flagged or blocked. However, these solutions were restricted to specific communication channels and could not provide complete device-level protection. Cloud-based machine learning approaches were also used, where URLs were sent to remote servers for analysis. While these methods improved detection accuracy, they introduced delays due to network dependency and raised concerns about user privacy and data security. Tools like VirusTotal required users to manually check URLs, making them less convenient and less effective in real-time scenarios.Overall, the existing systems lacked real-time monitoring, full coverage across all applications, fast response, and privacy-focused design. These limitations made them less effective in handling modern phishing attacks, which are more dynamic, fast-evolving, and capable of bypassing traditional security mechanisms.

1.4 Proposed Approach

1.4.1 Aim of the Project

The main aim of the project is to develop a mobile-based phishing detection system that can monitor user activity, analyze URLs, and classify them as phishing or legitimate in real time. The system is intended to achieve high accuracy while maintaining low latency and preserving user privacy by performing all operations directly on the device.

1.4.2 Motivation

The motivation behind this project arises from the rapid increase in phishing attacks and the limitations of traditional detection techniques. Existing systems often rely on blacklists or cloud-based analysis, which are either ineffective against new phishing websites or raise privacy concerns. Users are often unaware of malicious links and may unknowingly provide sensitive information. This creates a need for a smart and automated system that can detect phishing attempts instantly without requiring user intervention.

1.4.3 Proposed Approach

The proposed approach follows a multi-stage process. Initially, the system uses a VPN-based mechanism to automatically intercept URLs from network traffic. This allows continuous monitoring of user activity across applications. After capturing the URL, the system performs feature extraction by analyzing its structure, domain patterns, and lexical characteristics. These extracted features are then provided as input to a machine learning model deployed on the device using TensorFlow Lite. The model processes the features and predicts whether the URL is phishing or legitimate. Based on the prediction, the system generates real-time alerts to warn the user about potential threats.

1.4.4 Applications of the Project

The application of this project is mainly focused on enhancing mobile security. It can be used for real-time phishing detection during web browsing and application usage. The system can also be integrated into mobile security tools or used for manual URL scanning. Additionally, it can support cybersecurity research by providing a practical implementation of phishing detection techniques.

1.4.5 Limitations of the Proposed Approach

Despite its advantages, the proposed system has certain limitations. The accuracy of the system depends on the quality of the training data used for the machine learning model. The system may require regular updates to handle newly emerging phishing techniques. Since the model runs on the device, there may be a slight impact on performance, especially on low-end devices. Furthermore, highly sophisticated phishing attacks that use advanced obfuscation techniques may still be challenging to detect.

Overall, the proposed approach provides a balanced solution by combining automation, real-time detection, and privacy preservation, making PhishGuard an effective tool for protecting users against phishing attacks on mobile devices.

1.5 Objectives

The PhishGuard project is aimed at developing an efficient and reliable system for detecting phishing attacks on mobile devices. The objectives of the project focus on ensuring real-time detection, improving accuracy, and maintaining user privacy.The primary objective of the system is to automatically monitor and intercept URLs accessed by the user through a VPN-based mechanism. This enables continuous tracking of network activity without requiring manual input. Another important objective is to analyze the intercepted URLs by extracting meaningful features such as structural patterns and domain characteristics, which help in identifying suspicious links.The project also aims to implement a machine learning-based classification system that can accurately distinguish between phishing and legitimate URLs. By deploying the model directly on the device using lightweight technology, the system ensures faster processing and reduces dependency on external servers. This approach also helps in preserving user privacy, which is a key objective of the project.

In addition, the system is designed to provide real-time alerts to users whenever a phishing link is detected, allowing them to take immediate action and avoid potential threats. The project also focuses on creating a scalable and efficient solution that can be used across different mobile environments.Overall, the objectives of PhishGuard are centered on improving mobile security by combining automation, intelligent analysis, and real-time response to effectively prevent phishing attacks

1.6 Sustainable Development Goals (SDGs)

The PhishGuard project is aligned with **Sustainable Development Goal 9: Industry, Innovation, and Infrastructure**. This goal focuses on promoting technological innovation and building secure and resilient digital systems. PhishGuard contributes to this objective by developing an advanced mobile-based solution for detecting phishing attacks using modern technologies such as machine learning and real-time monitoring.The project enhances digital infrastructure by improving the security of mobile devices, which are widely used for communication and online transactions. By providing a reliable and efficient phishing detection system, it supports innovation in cybersecurity and helps create a safer digital environment. This contribution is important for strengthening trust in digital technologies and ensuring secure access to online services. (see Fig 1.1 Sustainable Development Goals)

SDG9: Industry, Innovation, and Infrastructure

Fig 1.6 Sustainable development goals [1]

1.7 Overview of project report

- **Chapter 1: Introduction** to the PhishGuard project including background of phishing attacks, project statistics, existing technologies used, proposed approach, objectives, alignment with SDG 9, and an overview of the report structure.
- **Chapter 2: Literature Review** of previous research and phishing detection techniques, focusing on URL-based analysis and machine learning approaches.
- **Chapter 3: Methodology** outlining the techniques and tools used such as VPN-based URL interception, feature extraction, and on-device machine learning using TensorFlow Lite.
- **Chapter 4: Project Management** covering the development timeline, risk analysis, and planning involved in building the PhishGuard system.
- **Chapter 5: System Analysis and Design** including system requirements, architecture, data flow, and overall design of the phishing detection process.
- **Chapter 6: Implementation (Hardware and Software)** describing the hardware and software requirements, development environment, and implementation details of the PhishGuard application.
- **Chapter 7: Evaluation and Results** providing testing methods, system performance, detection accuracy, and analysis of results.
- **Chapter 8: Social, Ethical, and Legal Aspects** discussing privacy, security concerns, and ethical considerations related to phishing detection.
- **Chapter 9: Conclusion and Future Scope** summarizing the outcomes of the project and

suggesting improvements and future enhancements.

- **References** listing all sources and materials referred to during the project.
- **Appendix** including supporting materials, additional data, and technical details related to PhishGuard.

Chapter 2

Literature Review

Phishing is a continuing cybersecurity threat since attackers will create fake URLs that have a strong semblance to actual domains to obtain sensitive users data. Even though this technique is also efficient and computationally cheap, it cannot detect newly created, or obfuscated phishing links, and thus cannot be used to address the evolving attack tactics. In order to conquer these weaknesses, heuristic-based models were also proposed and these structural features include length of URL, depth of subdomain and presence of suspicious keywords. Although the methods enhance the detection accuracy, they rely on a set of rules which can be outdated and can fail to give the correct classification when the attackers alter patterns.The recent developments focus on using machine learning in order to overcome these limitations. In contrast to rule-based systems, machine learning models are trained to recognize patterns based on data and evolve in response to emerging phishing schemes. In contemporary systems URLs are converted to numeric representation depending on lexical and structural characteristics thus allowing models to differentiate between legitimate and malicious links in a more effective manner. The implementation of this idea in the PhishGuard system is achieved by having a feature extraction pipeline where hostnames are analyzed and filtered on trusted domains before being processed. In case a domain has not been identified by a safe domain, the features are transmitted into a neural network model executed with the help of the TensorFlow Lite that enables quick inferences on-device and does not involve the use of cloud services. The training stage is when XGBoost is applied to improve the performance of learning, and the deployment is when the lightweight neural network model is applied to make a real-time prediction.Mobile environments bring more difficulties in detecting phishing because of fewer computational resources, and fewer visible URLs on smaller screens. Users are under greater vulnerability as mobile interfaces tend to conceal complete web addresses which puts them at risk associated with dealing with malicious links. Conventional desktop-based detection systems do not exist in such systems, particularly when requiring continuous connectivity to the internet or cloud computing. PhishGuard overcomes these limitations by offering a mobile featured solution, which works totally on the cell phone. It includes two prime ingredients: manual URL-scanning and automatic background protection.The system incorporates a VPN-based service that constantly checks outgoing network traffic, harvests information about the domains and appraises them prior to a connection being made. Such a proactive strategy is necessary to make sure that a possible threat is detected and prevented on-the-fly. Manual and automatic detection mechanisms have a common pipeline that involves trusted-domain filtering, feature extractions and neural network classification.

This design has reduced latency besides ensuring privacy of the user since there is no transmission of sensitive information to the outside.There remain gaps in terms of achieving accuracy, real-time performance and privacy within a single framework in spite of the very high advancement in terms of phishing detection. A number of the current systems are either dependent on fixed approaches or need to be processed in the cloud and thus, not efficient in the mobile devices. To overcome these constraints, PhishGuard combines dynamic machine learning with minimal on-device implementations and in-service, constant VPN-style monitoring to create an efficient, scalable, and privacy-conscious phishing detection engine.

Table 2.1 Summary of Literature reviews [1]

Application	**Detection Approach**	**Real-Time**	**Detection Stage**	**Key Feature**	**Limitation**
Google Play Protect	App scanning with limited URL analysis	No	After installation/execution	Built-in system protection	Not focused on phishing URLs
Avast+ Kaspersky	Signature+ cloud-based detection	Partial	During/after network activity	Strong threat database	Requires internet/cloud support
Bitdefender + Norton	Cloud+ behavioral+ blacklist ML	Partial	During/after page load	Behavior and blacklist detection	No pre-request blocking
Lookout Mobile Security	Cloud-based machine learning	Partial	After access	ML-based classification	No DNS-level interception
Sophos Intercept X	Heuristic URL filtering	Partial	During browsing	Rule-based detection	Limited real-time enforcement
NordVPN+ AdGuard	DNS-based blacklist filtering	Partial	DNS level (pre-request)	Blocks known malicious domains	Ineffective for new/unknown URLs
PhishGuard (Proposed)	URL-based ML + VPN integration	Yes	Pre-request (before connection)	On-device ML + real-time blocking	Requires VPN permission

Chapter 3

Methodology

3.1 Introduction to V-Model

The V-Model, also known as verification and validation model, refers to a process-oriented approach to software development in which a connection is made between all development phases and testing phases. This approach can be represented as a model in a "V" shape, where the left side consists of requirements analysis, system design, and implementation, while the right side consists of testing processes including unit testing, integration testing, system testing, and validation.

In this case, when implementing the PhishGuard system, the V-model guarantees that each of the developmental stages, from the design of URL feature extraction to mobile app deployment, will be tested to ensure that all development phases have been done correctly, and validation is guaranteed to ensure that the system will successfully detect phishing URLs.It would also be appropriate to apply the V-Model to PhishGuard since the application deals with cybersecurity issues, which require accurate and reliable operation and early detection of errors.

Basic V-Model Cycle Diagram

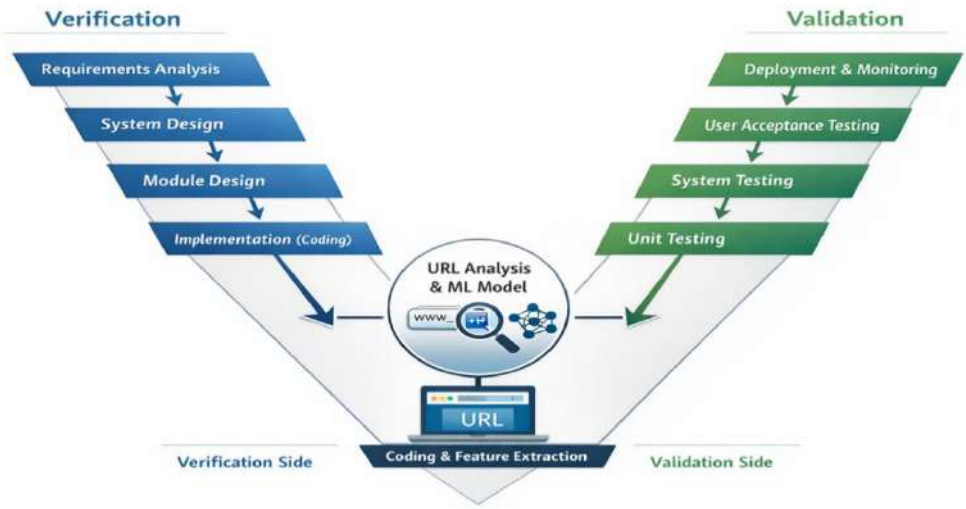


Fig 3.1 V-Model Methodology [2]

The V-Model diagram for the PhishGuard system represents a structured development approach where each development phase is directly associated with a corresponding testing phase. The left side of the diagram shows the **verification stages**, while the right side represents the **validation stages**, with coding placed at the bottom as the connecting point.

Checking Phase (Left Side)

The verification stage is aimed at constructing the system right as per requirements.

- **Requirements Analysis:**

At this step, the requirements of the system are clarified, such as phishing URL detection, monitoring based on VPN, real-time notification, and user verification. This will make the system objectives clear prior to development.

- **System Design:**
  The general design of PhishGuard is envisioned, such as the mobile application, a backend server, machine learning model, and a VPN monitoring service. The interplay of these elements is also well outlined.
- **Module Design:**
  The system will be separated into smaller modules, which include URL feature extraction, machine learning prediction, API communication, and notification handling. The modules are to have particular responsibilities.
- **Implementation (Coding):**
  This is the stage of the real system development. The mobile application is developed based on the Android technologies, whereas the backend is developed based on FastAPI and the machine-learning model trained and deployed with the help of the tensorflow Lite.

Central Phase (Bottom of V)

- **URL Analysis & Machine Learning Model:**
  This is the essence of the PhishGuard system. It entails deriving features out of URLs and classifying with the trained model as phishing or legitimate. It is a stage, which links development and testing.
- **Phase of validation (Right side)**
  The validation phase is important to ensure that the developed system is functioning properly and it satisfies the user expectations.
- **Unit Testing:**
  The separate modules like feature extraction, API calls and prediction logic are tested separately to ensure that they are right.
- **System Testing:**
  The entire system is put to test and it is ensured that all the components are functioning in unison such as the front end, back end and VPN monitoring.
- **User Acceptance Testing:**
  The system is also tested in a user perspective to guarantee the usability, accuracy and generation of appropriate alerts.
- **Deployment & Monitoring:**The last system is implemented on Android devices. The VPN service keeps track of network traffic, and the system offers real-time phishing detection and

alerts.

1. The V-Model is a fundamental concept of PhishGuard.
2. One of the main peculiarities of the V-Model is parallelism between the development and testing phases:
3. Requirements ↔ Acceptance Testing
4. System Design 2 System Testing.
5. Module Design/Unit Testing.
6. This makes sure that all the stages present are checked and tested, and the quality of systems can be enhanced.

3.2 Requirements Specification

The requirements specification stage establishes the functional and non-functional requirements of PhishGuard system prior to the commencement of development. This will make the system meet user requirements and security goals.

3.2.1 Functional Requirements

- Machine learning should be used to identify phishing URLs by the system.
- It should allow users to manually scan URLs.
- It must monitor network traffic automatically by using a VPN based service.
- There should be real-time warnings of suspicious links in the system.
- It must show a risk score to determine the level of the threat that is observed.
- Firebase should be used to secure user authentication.

3.2.2 Non-Functional Requirements

- The system must be capable of quick detection.
- It must use small things on the device.
- There should be privacy and data security of the users in the application.
- The interface must be easy to use and straightforward.
- The system should be scalable and adaptable to new phishing patterns.
- Effective requirement specification will guarantee that the system will respond to real-life phishing threats and not redesign any of the requirements without a need to do it.

3.3 System and Functional Design

3.3.1 System Architecture

The PhishGuard system is a modular and scalable system that has various interconnected modules that handle a given task to guarantee effective phishing detection.

3.3.2 Mobile Application (Frontend):

The mobile application, created in the Android Studio, is the main interface between the user and the

system. It enables the user to scan URLs manually, see the detection results and get alert messages about suspicious links. The interface is made user friendly and straightforward in order to ensure that the results can be comprehended by non-technical users. It also takes care of user sessions and communicates securely with the backend over API calls.

3.3.3 VPN Monitoring Service:

This is an element that is important and runs in the background. It captures network traffic on its way out of the device, at the device level, and pulls out URLs, or hostnames, before a connection has been established. In this way, it supports the real-time detection of phishing, without the need to interact with the user. This proactive method can make sure that users do not visit malicious websites.

3.3.4 Backend Server:

FastAPI, which is a fast and efficient API handling, is used to implement the backend. The mobile application sends URL requests to it, which process the requests and respond with prediction results. The backend will be in charge of communication between the frontend and the machine learning model as well as low latency and high performance.

3.3.5 Machine Learning Model:

The system is centered around the machine learning model. It is trained using a dataset of phishing and legitimate URLs and analyzes various extracted features such as URL length, domain structure, suspicious keywords, and patterns. The model, based on these features, classifies the URL and produces a risk score that represents the likelihood of the URL being phishing. The model is optimized and deployed with the help of TensorFlow Lite and is deployed efficiently on mobile devices.

3.3.6 Database and Authentication:

Firebase is employed in managing secure user authentication and data management. It makes sure that user data is stored securely and only the authorized users can access the application. This enhances the overall security and reliability of the system.Functional Design

The functional design describes how the system operates step-by-step to detect phishing URLs efficiently.

3.3.7 Workflow:

- The user can either type in a URL or simply click on a link in any application.
- The URL is captured before it is accessed by the VPN monitoring service or a manual scan. elenchanted URL is relayed to a backend server via an API request.
- Feature extraction is done by the backend to determine significant features of the URL.L.
- These attributes are inputted into the machine learning model to be analyzed.

- The model predicts the URL to be phishing or legitimate.
- An indicator of the severity of the threat is produced as a risk score.
- The outcome of the prediction is returned to the mobile application.
- In the event that the URL is found to be suspicious, the user is alerted about that.

Explanation:

The PhishGuard system can be used to detect phishing manually and automatically, which is functional. When a user makes contact with a URL, the system makes sure that the URL is identified either by direct entry or by the VPN monitoring service. This URL is effectively processed by the backend server and meaningful features needed to classify it are extracted. The machine learning model considers all these features and decides the type of the URL with a high degree of accuracy. The risk score that is generated assists the users to know the extent to which the link is dangerous, instead of merely classifying it as an unsafe or safe link. Lastly, the outcome is presented on the mobile app and notifications are sent where appropriate. This end to end process guarantees real time detection, rapid response and low user input.

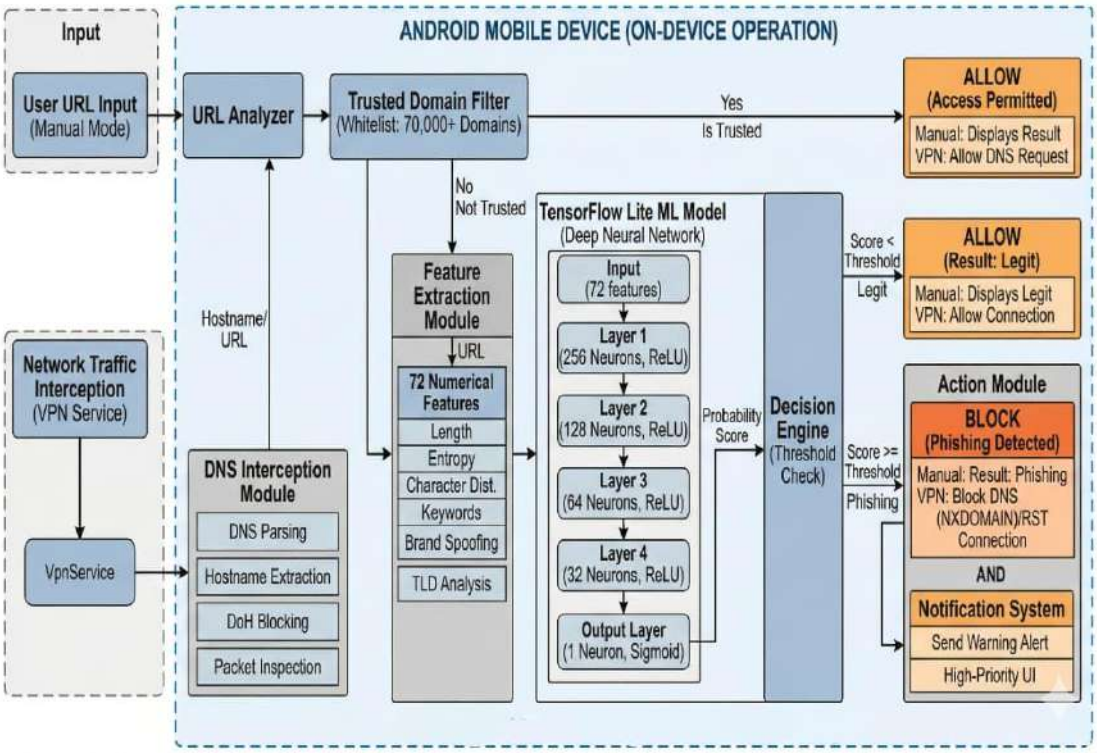


Fig 3.3 Block diagram of the Phishguard architecture [3]

The PhishGuard system design helps to identify phishing in real-time on an Android device both through manual entry and background checks. When in manual mode, the user just enters a URL, whereas in background mode, DNS requests are captured and hostnames are extracted by intercepting network traffic, using a VPN service.The URL is initially run through the URL Analyzer and the URL is subsequently compared with a Trusted Domain Filter which contains a whitelist of

known safe websites. In case the domain is trusted, it is immediately allowed to access. Otherwise, it is forwarded to the Feature Extraction Module, in which 72 numerical characteristics are created depending on the structure and characteristics of the URL. Such features are then transferred to a Neural Network model that is implemented in TensorFlow Lite, generating a probability score of whether the URL is a phishing or a legitimate one. This score is compared to a predefined threshold by the Decision Engine to categorize the URL. Based on the result, the Action Module either allows the connection or blocks it. The system also sends a notification in case of phishing detection in order to alert the user. This architecture will provide quick, effective and on-board phishing detection without the use of external servers.

3.4 Unit Design and Implementation

The PhishGuard system will be split into smaller functional modules to provide a more efficient development process, easy testing of the system, and easier maintenance. The modules are created to execute a particular task on its own and harmoniously with the other parts of the system.

3.4.1 Extractor URL Feature Module:

This module is charged with the responsibility of extracting meaningful features to the input URL that can aid in the detection of phishing patterns. It examines different features including the length of the URL, subdomains count, the use of special characters, IP addresses, and domain names, and the use of suspicious keywords. These features are extracted and then converted to a structured format which can be read by the machine learning model. This module has a direct effect on the quality of the detection system.

3.4.2 Machine Learning Module:

Machine learning module is the main decision-making part of the system. It trains a model (Random Forest) to classify URLs as phishing or legitimate based on the features extracted. The model is trained on a data set with safe and malicious URLs to get acquainted with the patterns and relationships. The trained model then produces predictions and a risk score of the likelihood of phishing. This module is performance optimized and is implemented in lightweight form to guarantee rapid execution.

3.4.3 API Module:

The API module enables a communication between the mobile app and the back-end server. It is deployed on FastAPI and offers using URL sending endpoints, receiving predictions endpoints, and response handling endpoints. The module provides safe and effective data transfer based on HTTP requests and JSON responses. It also controls the error management, request validation and response formatting to ensure the seamless interaction among the system parts.

3.4.4 VPN Monitoring Module:

This module allows real-time phishing to be detected by keeping track of outgoing network traffic on the mobile device. It runs as a background service and blocks the URLs prior to their being opened by the user. The URLs obtained are sent to the backend to be analyzed. This proactive monitoring system puts in place early detection of phishing threats even without human intervention, thus improving the security of the system.

3.4.5 Notification Module:

The notification module takes the role of notifying the users about the detected phishing threats. Once a URL is considered suspicious, this module will create an alert message and show it on the mobile device. The alerts should be simple to understand and informative, sometimes with the risk score and warning message. This assists users to do something at once and evade malicious links.

3.4.6 Implementation Approach

The modules of the system are executed on their own basing on the principles of modular programming. This can make codes easier to read and debug, and can entirely update or substitute individual modules without impacting the rest of the system. The interface is defined and guarantees ease of communication between the modules. Also, this design facilitates scalability, which allows enhancing it in the future through the addition of new detection methods or the enhancement of existing functionality.

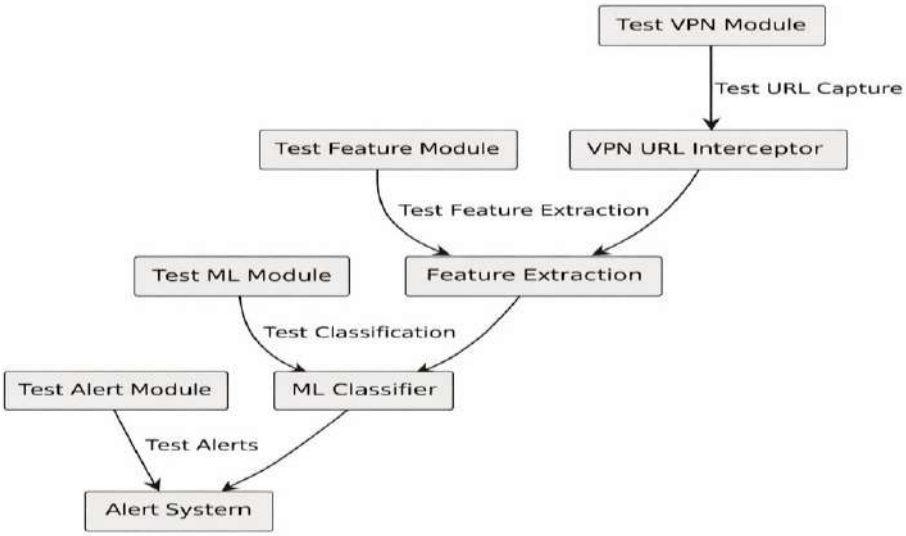


Fig 3.4 Unit Design Flowchart [4]

The PhishGuard unit testing diagram shows how each of the system parts will be tested to achieve proper and credible phishing reading. It starts with the Test VPN Module that checks the VPN service to be used on intercepting network traffic and capturing URLs. The VPN URL Interceptor is also tested to verify a proper extraction of hostnames and DNS requests out of the network data. Simultaneously, the Test Feature Module analyzes the Feature Extraction part and assesses whether it generates the necessary numerical features of URLs, such as structural and lexical features that are

important in classification, correctly.These features are then fed into the ML Classifier where the Test ML Module is used to make sure that the machine learning model is properly classifying URLs as legitimate or phishing. Lastly, the Test Alert Module will test the Alert System to ensure that correct warnings are issued whenever phishing is detected. This unit testing methodology will guarantee that every module, such as VPN interception, feature extraction, classification and alert generation, works properly on its own, and thus, enhances the reliability and robustness of the PhishGuard system as a whole.

3.5 Integration Testing

Integration testing confirms the interaction of the different modules of PhishGuard system after unit testing. It makes sure that all the parts work as a whole system and data are properly passed through various levels of the architecture.

3.5.1 Integrated Modules

- Mobile Application and Backend API.
- VPN Monitoring Module and URL Processing.
- Machine Learning Model and Feature Extraction Module.
- Machine Learning Model and Result Processing
- Notification System and Backend Server.
- Firebase Authentication and Mobile Application
- Freebase Authentication and Mobile Application

3.5.2 Integration Process

- The user triggers a scan of the URL or clicks on a link.
- The URL is either captured manually or by VPN monitoring module.
- The app for mobile will relay the URL to the back-end through API.
- The URL is extracted of features by the backend.
- Features extracted are fed to the machine learning model.
- The model generates a prediction along with a risk score.
- Output is processed by the backend and response is given.
- The mobile application shows the outcome and creates alerts when needed.

Explanation

In integration testing, all modules that have been tested before are put together to ensure the end-to-end functionality of the system is tested. The testing will be focused on the seamless communication between the machine learning model, background server and the mobile app. It also verifies the VPN monitoring module is functioning properly since it feeds live URLs into the processing pipeline and the notification system responds to the presence of a threat. Flaws in data flow, slow response or

integration errors are identified and resolved at this stage so that the system will be able to operate reliably.

3.5.3 Testing Focus Areas

- Data consistency and flow between modules.
- API communication and response handling.
- System latency and performance.
- Error handling and recovery mechanisms
- Detection and alert generation in real-time.

3.6 Verification and Validation

The verification and validation processes are significant to verify the accuracy, functionality and reliability of the PhishGuard phishing detection system. Verification provides an answer to this question, "Are we building the product right? It is concerned with the process of making sure the system is built as per the requirements and design. Verification in the PhishGuard system entails testing the appropriate functionality of modules including URL feature extraction, machine learning model integration, backend API communication, and VPN monitoring service. Terminal code level testing and module level checking is undertaken to make sure that every component is giving the desired output. This aids in detecting mistakes at an early stage and makes sure that the system is running as it is supposed to be during development.

Validation answers the question, "Are we creating the correct product? It makes sure that the system satisfies user requirements and how well it works in actual conditions. The approach in PhishGuard is that the system is validated using real phishing and legitimate URLs. Manual scan and VPN-based automatic scan are tested to confirm that the system properly detects malicious links and provides proper alerts. Response time, accuracy and usability of the system are also tested to ensure that the system offers reliable real time protection to the users.

3.6.1 Techniques for System-wide Verification and Validation

- Verification: Requirement checking, module-level testing, API validation and verification of the feature extraction and machine learning prediction output.
- Testing: Functional testing with real world URLs, performance testing, real time testing with VPN monitoring and user level testing on the alert accuracy and usability of the alert.
- The difference between the verification and validation makes sure that the PhishGuard system is not only implemented properly but it actually works. Verification assures that every module is functioning according to plan whereas validation assures the entire system to provide correct phishing detection and user protection. This joint strategy enhances the quality, reliability, and overall performance of the system.

3.6.2 Benefits, Challenges, Tools, and Technologies Used

- V-Model methodology will offer a systematic way of developing the PhishGuard system and there will be proper coordination in development and testing.
- Definitive Development Structure: A development stage will be associated with a testing activity and this will enhance the accuracy of the implementation.
- Improved Requirement Alignment: Makes sure that the system does fulfill its main purpose of detecting phishing by maintaining design and development.
- Better Quality Assurance: Errors on the way to development are better identified, which results in a stable system.
- Effective Error Detection: Assists to detect errors within the modules like URL processing, prediction logic and VPN monitoring.
- Improved System Reliability: Provides stable operation of the system during manual scanning and in automatic protection mode.

3.6.3 Difficulties and countermeasures.

- Low Flexibility: It is hard to make changes in later stages.
- Mitigation: Requirements were clearly defined at the beginning and reviewed when necessary.
- Integration Complexity: Integrating Android app, VPN service, and ML model may cause problems.
- Mitigation: Modules were incorporated in a step-by-step fashion with constant testing.
- Performance Constraints: Performance with real-time detection can impact the performance of the device.
- Mitigation: Lightweight TensorFlow Lite model and optimized processing were used.
- Testing Effort: There are several testing phases that will take more time.
- Mitigation: Special attention was paid to such critical modules as detection accuracy and VPN monitoring.

3.6.4 Used Tools and Technologies.

- Frontend (Mobile Application)
- Android Studio: The Android application is developed and tested in this.
- Kotlin: Used for implementing application logic.
- XML: XML is utilized in developing the user interface layouts.
- Android SDK: Offers app development tools and libraries.
- Authentication and Cloud
- Firebase Authentication: This is employed to authenticate users who will be secured and

manage their sessions.

3.6.5 Machine Learning

- TensorFlow Lite: On-device phishing.
- Scikit-learn: This is to train the phishing detection model.
- Pandas and NumPy: applications in handling and preprocessing data sets.

3.6.6 Core System Components

- Android VPN Service: This is used to track and intercept network traffic that goes out in order to automatically identify phishing.
- Notification System: This will be used to notify users of phishing threats in real-time.
- Local Storage (Room / SharedPreferences): Used to store scan history and user settings.
- Tools of development and testing.
- ADB (Android Debug Bridge): Testing and debugging the application.

Chapter 4

Project Management

4.1 Project Timeline

The timeframe for the PhishGuard app development project was set from January 3 to April 28. The Gantt chart was selected as the major tool to visually depict the durations, dependencies of project tasks, meetings, and milestone events. It shows each project task as a line graph against time. This pictorial representation is definitely in keeping with a progress check and makes sure the development goes on in a well-disciplined way.Project Gantt chart is a sort of control panel where the project members can see a clear picture of the work they do, discover the tasks overlapping and also keep a logically ordered set of tasks from the planning to the implementing stage. Using a calendar, the working days were initially distributed monthly for the project activities and allocation of resources. Besides that, the major points such as requirement approval, system design, coding progress, and testing of the final product were also regularly monitored.

Table 4.1.1 project planning timeline [2]

Task No.	Activity	Start Date	End Date	Duration (Days)	Milestone/Deliverable
P1	Project topic confirmation & problem definition	03-Jan	07-Jan	4	Problem statement approved
P2	Literature review & study of phishing techniques	08-Jan	15-Jan	7	Literature summary completed
P3	Requirements analysis & system specification	16-Jan	22-Jan	6	Requirements finalized
P4	System design (architecture & modules)	23-Jan	31-Jan	8	Design completed
Milestone M1	Planning Phase Completed	-	-	-	All design documents approved

The Gantt chart served as a dual-purpose tool - it was used for planning, and at the same time, it was a tracking tool as it provided a glance view of the whole project. It was very instrumental in ensuring that all the phases were completed on time and also facilitated continuous monitoring through monthly reviews. The development of PhishGuard was divided into two major parts: project planning and project implementation. During the planning phase, the team focused primarily on problem analysis, a survey of existing phishing detection techniques, getting user requirements, and designing the system architecture. The work was completed in January.

The implementation phase succeeded the planning phase and lasted from February to April. It

consisted of developing the VPN-based URL interception module, implementing the feature extraction process, integrating the machine learning model, and setting up the alert system. The work was arranged very logically, and development was done in the form of monthly reviews at the start of each month, aiming at ensuring the right progress and making improvements. Later-on, we conducted testing activities such as unit testing, integration testing, and system evaluation. The project ended with the documentation and preparation of the final report before submission.

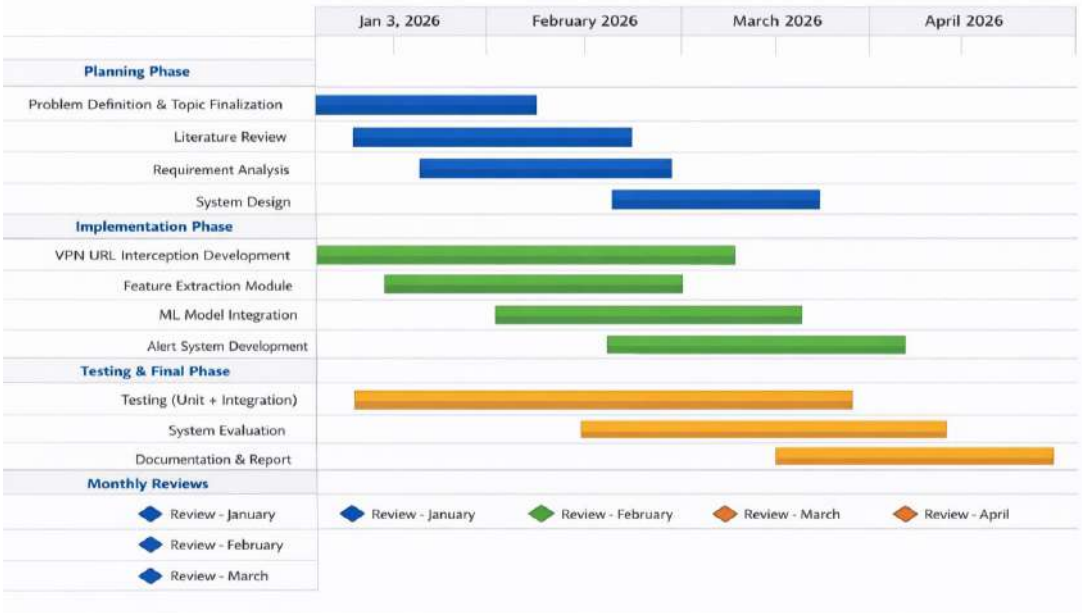


Fig 4.1 Timeline Gantt Chart [5]

Table 4.1.2 Project implementation timeline [3]

Task ID	Activity	Start Date	End Date	Duration (Days)
I1	VPN-based URL interception development	01-Feb	10-Feb	10
I2	Feature extraction module implementation	11-Feb	20-Feb	9
I3	Machine learning model integration	21-Feb	05-Mar	13
I4	Alert system development	06-Mar	15-Mar	9
I5	Testing (unit & integration testing)	16-Mar	05-Apr	20
I6	System evaluation & optimization	06-Apr	12-Apr	6
I7	Documentation & report preparation	13-Apr	20-Apr	7

4.2 Risk analysis

Every project comes with a certain level of risks that can potentially derail the project development, change the expected results, or even the time required for finishing the project. Understanding risks is a great means of problem identification in a problem-solving process - first finding out about the

problems, next judging how likely they are and what/ how big their impact is, last but not least deciding the reducing the effects of the most significant problems ways. PhishGuard project was taken into consideration from the three points of view: technical, operational, and scheduling. PESTEL (Political Economic Social Technological Environmental, and Legal) was the method applied to find the internal and external factors that might affect the project. Among the technological risks, one was singled out as the most significant one because of the reliance on machine learning models, real-time URL interception, and mobile device limitations. Apart from data quality, model prediction, system performance, and real-time processing were majorly other risks identified. Since the system is a learning machine, it is very important to have a high-quality and well-balanced dataset in order to produce accurate results. A poor-quality or highly unbalanced dataset can cause a phishing URL to be labeled incorrectly.

One of the key issues that must be addressed, is the resource limitation of mobile devices. Implementation of machine learning models and a VPN-based continuous monitoring system might negatively impact device performance and increase battery consumption. Uncontrolled, these factors combined with the reliance on the uninterrupted integration of various modules, e.g. URL interception, feature extraction, and classification, would also lead to risks exposure from the operational point of view. A delay in one module could potentially cause the entire chain of project milestones to fail. The system accesses users' browsing data, so security and privacy concerns are the first to be considered. One of the measures the project adopts to mitigate the risk is running a sparse machine learning model via TensorFlow Lite and at the same time, the mobile device running time is hugely optimized. Furthermore, the system is thoroughly inspected and fine-tuned on a continuous basis not only to enhance accuracy but also to lessen system load. Besides, the modular design and regular testing make it easier to locate errors and subsequently implement fixes.

Table 4.2 Project phase risk matrix [4]

Project Phase	**Risk Level**	**Description**
Planning Phase	Low	Risks are minimal as activities involve problem definition, literature review, and requirement analysis. These are managed through regular discussions and proper documentation.
Design Phase	Medium	Moderate risks due to system architecture design and module planning. Errors in design may affect later stages, but can be controlled through reviews and validations.
Implementation Phase	High	High risk due to development of VPN interception, feature extraction, and ML integration. Technical issues and module dependencies may impact progress.
Testing Phase	High	High risk as system accuracy, performance, and real-time detection are validated. Bugs or incorrect predictions may require

		rework.
Deployment & Documentation Phase	Low	Low risk since the final system is already tested. Activities mainly involve report preparation and final submission.

4.3 Project Budget

PhishGuard project had no devoted financial resources. The application was built using only open-source software and other resources that were accessible free of charge. At no point during the project cash funding, or investment was received. All team members equally shared the involved time, effort, and technical knowledge. Planning, coordination, and frequent reviews; these three elements ensured that the project was completed without any problems in the given time. The PhishGuard program was implemented without any monetary expense. The development process was performed only on open-source software and free resources. No external funding or budgeting was necessary. Sharing of design development testing, and documentation phases was among the equal contribution of all team members. Time and effort were the only resources that the project had to invest. PhishGuard project is a demonstration that proper planning and teamwork can lead to a successful outcome even when no financial budget is available.

Chapter 5

Analysis and Design

The system is built in such a way that it has a modular architecture, which incorporates Android frontend, background VPN service, and an on-device machine learning model. The successive user interactions, network traffic, and URL entering are distinguished as the important data sources during analysis. The architecture guarantees real-time URL extraction, feature processing and classification with a TensorFlow Lite model. Firebase authentication allows controlling access to users, and effective information sharing among elements allows the system to remain constantly protected. The model is low-latency optimized and gives high-precision phishing detection with minimal resource usage on mobile devices.

5.1 Requirements

5.1.1 System Requirements Overview

This step involves determining the PhishGuard system requirements. The system requirements are as follows Phishing URLs should be detected in real-time using mobile devices.

System Hardware (HW) Requirement Phase

- The discovery of the first requirements of the system: Android device with minimum 2 GB RAM and the possibility to work with the recent Android OS.
- The discovery of the input parameters of the system: URLs that the user opens via browsers and applications.
- The identification of the system results: Detection of legitimate/phishing URLs and sending alerts.
- The determination of the system relations: Mapping of features of captured URLs.
- System Software (SW) Requirement Phase: The discovery of the constraints of the system: Limited processing power, limited life, and performance of mobile devices.

System Software (SW) Requirement Phase

- Initial Conditions: The installation of PhishGuard and the activation of the monitoring system based on the VPN.
- Input Parameters: URLs and features that are mined by the application.
- System Outcomes: Real-time Phishing identification.
- Relations: Features extracted, machine learning, and relations with the VPN module.
- System Constraints: Effective utilization of resources in the mobile device.

5.1.2 Additional Requirements

Data Collection: Automatic collection of URLs by the application with the help of a VPN-based

interception system

Data Analysis: Extraction and classification of features by machine learning model.

System Management: Monitoring of the system in the background and efficient use of resources

Security: Efficient use of resources on the mobile device for the sake of privacy and prevention of leakage of data

User Interface: The user is notified to efficiently detect phishing websitese.

5.1.3 System HW Design Phase

The PhishGuard app hardware system design has been guided mostly towards the need to make it mobile friendly and consume resources minimally. As an application to the usual Android smartphones, it has no dependence with any significant hardware specification or element. The ARM-based processor with at least 2 GB RAM will be needed to run the application and the embedded machine learning model. The app, local data storage and the TensorFlow Lite model will occupy approximately 200 MB of space. Internet is required in case a user wants to be verified by Firebase or perform a synchronization with the cloud (this feature is a cloud one and it is not mandatory). The detection aspect of the core phish of the app might also be effective without an internet connection. Hardware supporting at least Android version 8.0 is required such that the device can access numerous services such as VPN-based traffic monitoring and access notifications. On top of that, it will be able to continue running while held in the background and it won't be a source of power waste for the user who in turn, will always be protected against phishing. The development and testing is done on rather mundane computing systems like laptops or desktops.SharedPreferences is the application that controls the storage of local data. The whole project is maintained in version control using Git and GitHub and it is built using Gradle. Such software architecture can be used to detect phishing in a mobile setting accurately, safely and in real time.

5.1.4 Phase of SW Design.

The system software architecture of PhishGuard is the only one to integrate mobile app development, machine learning, and cloud services. All the way to the Android app it is developed using the Android Studio and Kotlin as the primary language, with the UI currently being designed in XML. Android SDK also includes additional features like permission management, background services, and system-level access like monitoring notifications or traffic interception using VPN, in addition to the basic API set. Firebase Authentication is utilized to verify the identity of the user, to secure it, and to manage the user session; Cloud Firestore is utilized to store user data and scan outcomes (optionally). The phishing detection algorithm is based on a machine learning model which was developed with the help of Python frameworks such as TensorFlow, Scikit-learn, and XGBoost. The model is subsequently condensed with TensorFlow Lite and deployed into the device where it does

fast and offline prediction. Besides BroadcastReceiver, NotificationListenerService, and VpnService which are the most significant Android components, the ones that are also applied to track SMS messages, notifications and network traffic.

5.2 Block diagram

The functional system diagram of PhishGuard has shown the process of real-time detecting of a phishing system on a mobile phone. The system starts with URL interception VPN whereby the user requests or network traffic are intercepted. The resulting URL is then forwarded to the feature extraction module, which examines features like length of URL, domain specifics and character configurations.These characteristics are analyzed with a lightweight machine learning model that runs on the device to either label the URL as phishing or legitimate. Depending on the classification, the system sends out an alert of phishing URLs or tags the URL as safe. Every processing is done locally on the device with quick response and increased user privacy without transferring data to other servers.

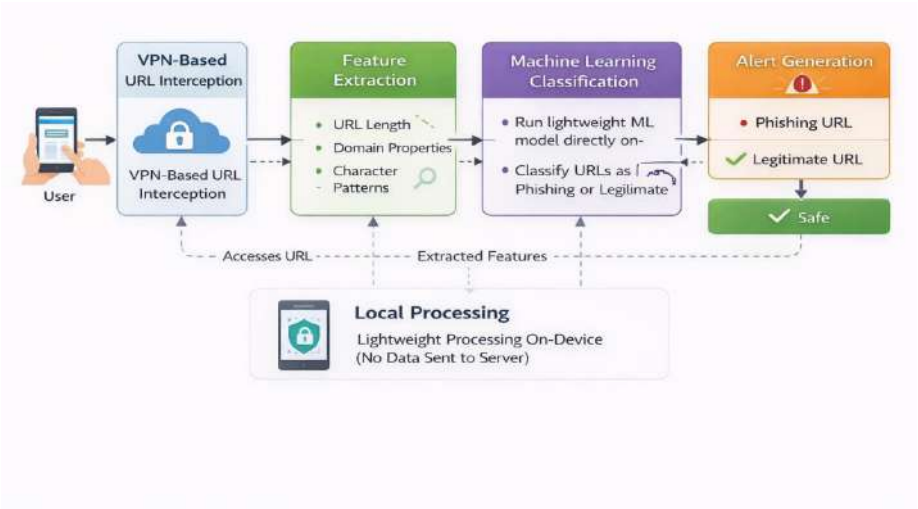


Fig 5.2 Functional block diagram [6]

In the center of the diagram are the users who can be represented by the mobile applications where they can enter the URLs manually or automatically collect data through the background services of VPN, SMS receiver and notification listener respectively.The next step is actually URL processing that involves a feature extraction stage and subsequently transmitting the features to the machine learning module (TensorFlow Lite) that is in charge of scoring the vulnerability of URLs with the phishing URLs scoring the highest risk.This risk score is then used by the decision engine alone or used together with other aspects to determine the URL to be safe, suspicious, or phishing.

5.3 System Flow chart

System flowchart of PhishGuard clearly and simply illustrates how the application works step by step.The very beginning is the user, who can either directly use the app or can also automatically

invoke the system when they get a message, notification, or even when they get a web request. The process initiating the app accesses the URL when the app is launched, and the URL is extracted via different means such as SMS, notifications, or through the web traffic intercepted by the VPN service.Upon acquiring the URL the next thing is the feature extraction which involves system analysis and finally, the user is informed about the result through alert messages or a dashboard. The system at times logs the result on a local machine to be referred to or analysed in future.In short, the flowchart is an excellent demonstration of the fact that PhishGuard is not only a service but a live platform that not only monitors the users but also analyses and secures them by making security decisions based on the raw data in real-time.

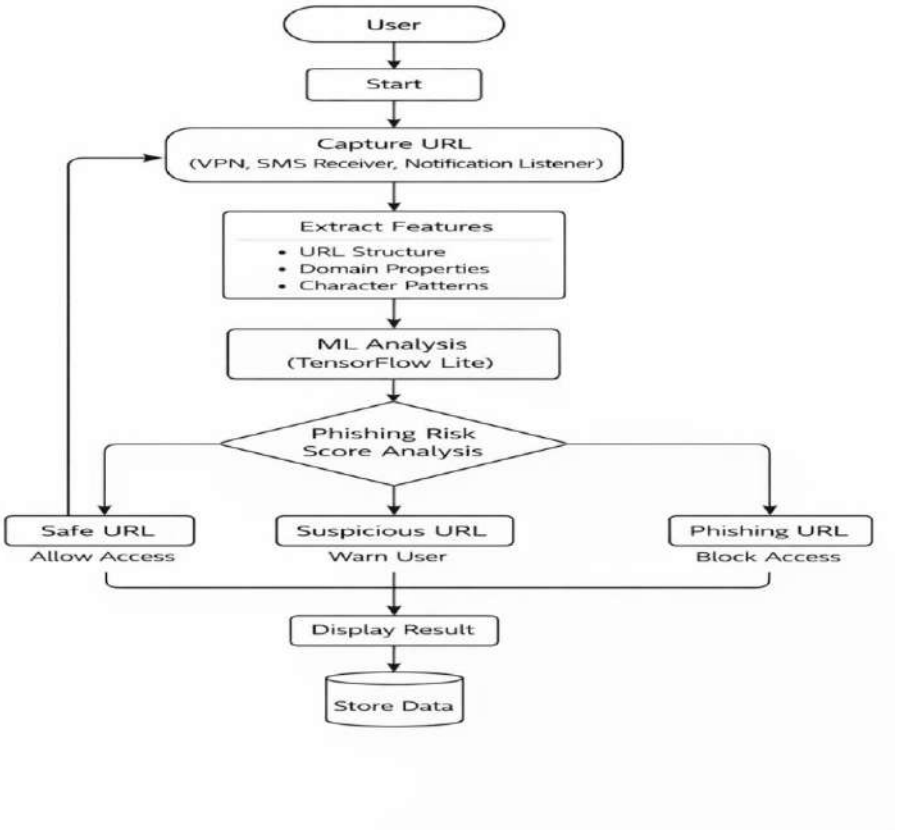


Fig 5.3 System flow chart of phishguard [7]

5.4 Hardware Components and Specifications

Table 5.4 Hardware Components and Specifications [5]

Component	**Specification**
Target Device	Android Smartphone
Processor	ARM-based Processor (Quad-core or higher)
RAM	Minimum 2 GB (4 GB Recommended)

Storage	Minimum 200 MB Free Space
Network	Wi-Fi / Mobile Data
OS Requirement	Android 8.0 (API 26) or higher
Sensors	Not Required
Development System	Laptop/Desktop (for coding and testing)

5.5 Software Components and Requirements

Table 5.5 Components and Requirements [6]

Component	**Specification**
Development Tool	Android Studio
Programming Language	Kotlin (Logic), XML (UI Design)
Framework	Android SDK
Machine Learning	TensorFlow Lite
Authentication	Firebase Authentication
Database	SharedPreferences, Cloud Firestore (optional)
Background Services	VPN Service, BroadcastReceiver, NotificationListener
Build Tool	Gradle
Version Control	Git, GitHub

5.6 Functional Modules of the PhishGuard System

Table 5.6 Functional Modules of the PhishGuard System [7]

Module Name	**Description**
Presentation Module	Handles UI like login, dashboard, alerts
Authentication Module	Manages user login using Firebase Authentication
Application Core Module	Controls overall app workflow and coordination
Automatic Protection Module	Uses VPN to monitor real-time network traffic

Manual Scanning Module	Allows users to scan URLs manually
URL Processing Module	Extracts features from URLs
Machine Learning Module	Predicts phishing using TensorFlow Lite
Decision Engine Module	Classifies URL as safe, suspicious, or phishing
Notification Module	Displays alerts and warnings to users
Data Storage Module	Stores data locally using SharedPreferences
Cloud Sync Module (Optional)	Syncs data using Firebase

5.7 Datasets and data resources

Table 5.7 Datasets and data resources [8]

Dataset Name	**Description**
UCI Phishing Websites Dataset	Contains URL-based features for phishing detection
PhishTank Dataset	Real-world phishing URLs and attack information
Generated Feature Data	Extracted features from URLs during runtime
User Scan Data	URLs scanned and results stored locally

5.8 Functional Modules and Brief Features

Table 5.8 Functional Modules and Brief Features [9]

Module Name	**Key Features**
Presentation Module	User-friendly UI, dashboard, alerts
Authentication Module	Secure login, session handling
Application Core Module	Controls scanning and protection flow
Automatic Protection Module	Real-time URL interception using VPN
Manual Scanning Module	User input-based URL scanning
URL Processing Module	Feature extraction and preprocessing

Machine Learning Module	Fast on-device phishing detection
Decision Engine Module	Risk scoring and classification
Notification Module	Instant alerts and warnings
Data Storage Module	Local storage of scan history
Cloud Sync Module	Optional cloud backup and sync

The combination of hardware and software tables outline the major technological pillars of the PhishGuard system. As far as hardware specs are concerned, the app can be installed and used without any difficulties on regular Android phones and no extra hardware is needed, so it is very user-friendly and low on resource consumption. Further, the software side shows that the creators of PhishGuard made a choice of the top and technologically advanced tools, namely Android Studio Kotlin Firebase, and TensorFlow Lite so that the developers are the most productive but users remain safe and have a fast experience even if the phishing detection is device dependent. The table of functional modules shows how different parts of the system perform different roles e.g. user authentication management, data processing, machine learning analysis, and decision making. By partitioning like this, the system is highly compact, easy to maintain and scalable.The datasets and data resources table highlights the role of both training and testing data in developing a model for phishing detection with high accuracy. The last table, functional modules and features, serves as a great reference to what responsibilities each module carries out, effectively connecting system architecture with actual operations. Together, these tables not only unravel the construction of PhishGuard but also its running process showing how a combination of different technologies and components collaborates to offer real-time phishing protection. In the end, the flowchart gives an explicit explanation of the mechanism of PhishGuard that includes continuous monitoring, analyzing, and safeguarding of users in real time by the transformation of raw inputs into security decisions that are both relevant and effective.

5.9 Selection of Devices and Software Comparison and Justification

- The efficiency, compatibility, and demands of real-time phishing identification are factors that guide the choice of devices and software to be used in the system, PhishGuard. Android is selected as the operating system because Android smartphones are ubiquitous and because they provide good support in terms of system-level services. The application is optimized to run on mid-range hardware with at least 2GB RAM, and will make good performance

without exceptionally fast hardware.

- Android studio is taken as the major environment in order to develop something since it contains inbuilt codes and features to develop, debug and design the interface. Kotlin is chosen in comparison with Java because it has the simplified syntax with more safety features and the potential for an asynchronous programming in modern times.
- In the case of machine learning, we use TensorFlow Lite that powerfully supports on-device detection of phishing without using continuous Internet access. This provides quicker response and better privacy in connection to cloud-based models.
- Firebase Authentication is used to manage users safely and easily without a custom backend. Android vpnservice is used to modify the web traffic and to then divide the traffic and extract the URLs to be analyzed and this is in line with the objective of the system to detect in real-time.
- Moreover, version control and collaborating are managed with the help of GitHub and properties and dependencies are automated with the help of Gradle to facilitate the development and launching of applications.

Table 5.9 Selection of Devices and Software Comparison [10]

Component	Selected Option	Alternative	Justification
Platform	Android	iOS	Better system-level access (VPN service)
Language	Kotlin	Java	Safer, concise, modern features
ML Framework	TensorFlow Lite	Cloud ML APIs	Works offline, faster inference
Authentication	Firebase Auth	Custom Backend	Secure, easy integration
Traffic Capture	VpnService	Browser-only APIs	Captures URLs system-wide
Version Control	GitHub	Local Git	Collaboration & backup
Build Tool	Gradle	Manual Build	Automation & dependency handling

5.10 Designing units

The PhishGuard system design has followed the appropriate standards for Android development for consistency, security, and performance. The programming language for the PhishGuard system is Kotlin. The appropriate programming conventions have been followed to avoid errors and make the programming process easy and convenient to read. The appropriate XML layout standards have been followed for designing a consistent and user-friendly interface for all the screens of the system. The system design has followed the appropriate modular programming standards with low coupling and high cohesion for the system components. The appropriate secure programming standards have been followed for the system, especially for authentication and data handling. The appropriate permission

standards have been implemented for Android runtime permissions for the privacy and security of the users and the system. The appropriate error handling standards have been implemented for making the system more reliable and convenient for users. The appropriate performance optimization standards have been considered for making the system run smoothly on mobile devices with low hardware specifications.

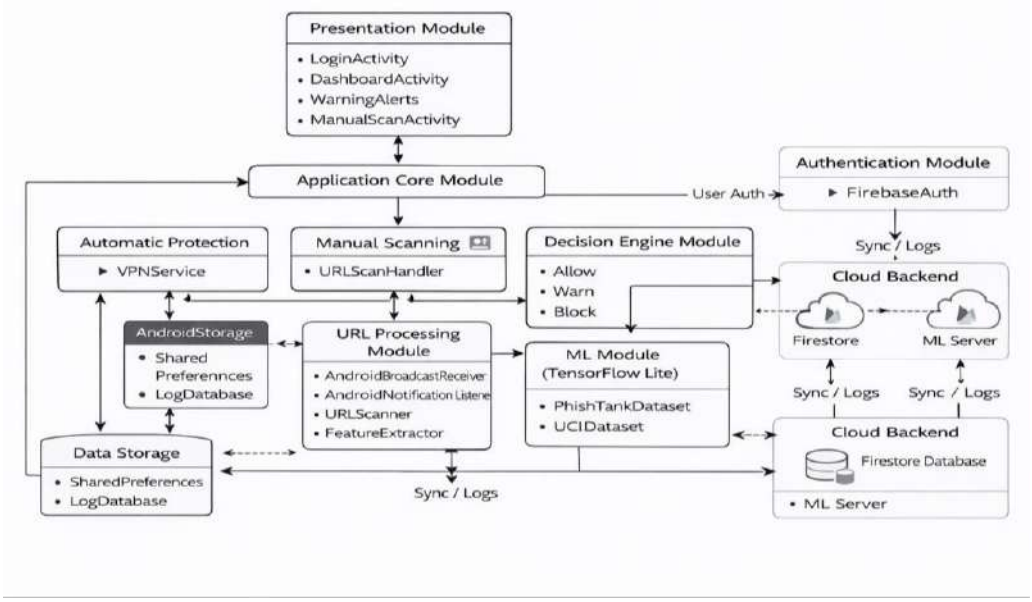


Fig 5.10 Unit Diagram of Phishguard [8]

5.10.1 Standards

The process of developing the PhishGuard solution is carried out in compliance with software engineering standards and Android development standards.

Guidelines and best practices for the structure of the application, UI and resource management are applied in the Android development process. The principles of Material Design are utilized to ensure consistent UI experience.Best practices for secure code development are used in order to safeguard users' information and reduce risks. RESTful API design is applied when necessary.Machine learning solutions within the system comply with machine learning best practices in terms of data processing and model optimization using TensorFlow Lite.

5.10.2 Domain Model Specification

The domain model of the PhishGuard system focuses on key entities and relations between them.Firstly, we need to consider the User entity which makes use of our system via the mobile application. This means that users are capable of interacting with the application (e.g., logging in to the application, scanning URLs, and obtaining results).URL is the input data that serves as input data in our system and includes information about extracted features and classification result.

Detection System comprises several modules such as feature extraction, machine learning algorithm, and decision engine responsible for processing data and providing results. The Result refers to the

entity containing the result of classification including different categories (safe, suspicious, or phishing). There is also an entity called Notification which provides notifications to users regarding their detection.

Furthermore, we should pay attention to Local Storage entities which serve as storage for scans' history and user preferences, along with Firebase Authentication for logging in.Therefore, our domain model allows us to depict the system's structure.

5.11 Design Models

The design of the PhishGuard system's unit is based on the modular approach, where all units of the system's components are individually designed based on a specific role to be performed. The role of the presentation unit is to cover all the interaction processes with the user, such as login. The role of the application core unit is to act as a controller of all the processes of communication that take place in the system. The role of the monitoring unit is to cover all the processes of collecting data from SMS messages and notifications, as well as internet traffic from system services such as BroadcastReceiver, NotificationListenerService, and VPN service. The role of the URL processing unit is to retrieve key features from collected URLs and send the features to the machine learning unit. The role of the machine learning unit is to process data from TensorFlow Lite. This unit provides a risk score for the URLs. The role of the decision unit is to determine the type of URLs collected by the system, such as safe URLs, suspicious URLs, and phishing URLs, based on a risk score provided by the machine learning unit.

The design models that have been used in the development of the PhishGuard system have provided a clear illustration of the structure of the system. The use of the layered structure has provided a clear illustration of the division of the Android app from the backend services. The functional model has provided an illustration of the flow of the process of the operations from the input of the data to the output of the result to the user. The use of the modular model has provided an illustration of the division of the system into different modules, such as the monitoring module. The use of the data flow model has provided an illustration of the process of the flow of the data from the URLs and the different stages it goes through. The use of the behavioral model has provided an illustration of the behavior of the system in response to real-time events, such as sending SMS messages and notifications from the web. The use of the deployment model has provided an illustration of the deployment of the system on the Android device and the cloud for authentication and syncing. This has provided a complete illustration of the system.

5.11.1 Communication Model

- PhishGuard’s communication model describes how information flows among various parts of the program.

- The PhishGuard system uses mainly internal communication because most operations happen locally. The mobile app sends information to internal components like the PhishGuard VPN service, URL processing, and machine learning models using data flow.
- If a URL is caught through manual input or the PhishGuard proxy service, the information is passed to the feature extraction algorithm. Features extracted are then sent to the machine learning classifier, and predictions are sent back to the decision engine.
- Communication with the outside world is minimal except for authentication, where necessary, and Firebase database operations, when needed. This avoids overreliance on remote machines and maintains privacy.
- The communication model guarantees smooth, low-latency communication between components.

5.12 IoT Deployment Model

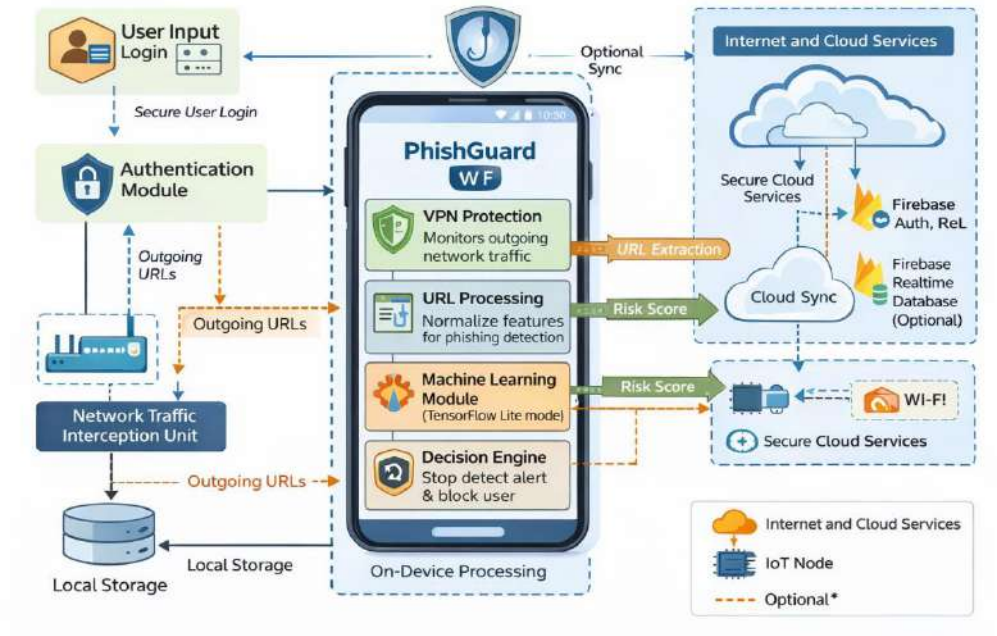


Fig 5.12 Iot deployment model of phishguard [9]

The PhishGuard software is able to be regarded as a lightweight IoT solution, which means that mobile phones act as smart IoT nodes in an interconnected system. The proposed deployment strategy is oriented towards real-time data processing, local computations, and reliable data communications.

Within the deployment model under consideration, the role of the IoT node is taken by the Android smartphone, which continuously monitors the outgoing network traffic provided by the virtual private network. URLs are extracted from outgoing traffic and analyzed using the on-device machine learning model built upon TensorFlow Lite technology.

The deployment model uses an edge computing solution, where data computations and phishing

detection are performed by the device itself. It helps reduce the amount of time needed to detect potential threats and enhance privacy and security by minimizing cloud communications.

Optional cloud connectivity can be provided using the Firebase platform, allowing for secure user identification and data synchronization. Nevertheless, basic functionality of the application does not depend on any cloud solutions.

This deployment model provides high flexibility, scalability, and effectiveness since every IoT node works separately, but still can connect to the cloud in case of necessity.

5.13 Functional View

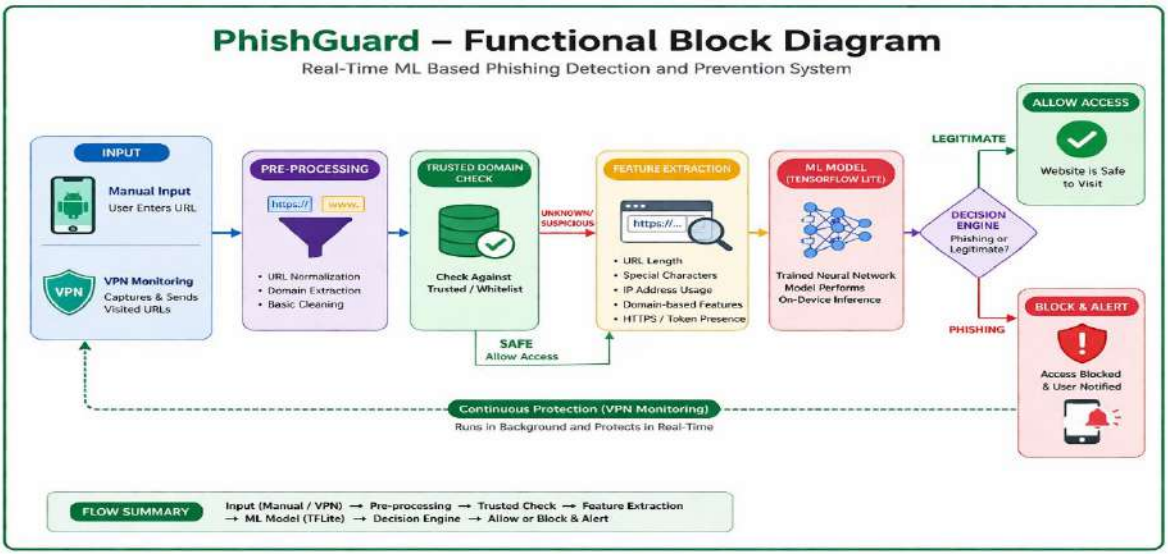


Fig 5.13 functional view of phishguard [10]

Functional approach of PhishGuard system highlights interactions between various system components needed for achieving its goals of conducting phishing checks and protecting end users. Functional approach is focused on describing operational processes and the functionality of specific modules. PhishGuard system offers two basic functionalities – URL scanning and real-time URL analysis. In case of URL scanning, user provides an Internet address and it gets analyzed by feature extractor module, then machine learning models process it. The result of classification is presented to the user as "safe", "suspicious" or "phishing".

The second functionality is provided automatically and it consists of monitoring the outgoing network traffic of an application and capturing URLs. Captured URLs also go through the feature extraction process and TensorFlow Lite model analyzes them and decides whether a website is safe or not. If site is recognized as safe, connection gets allowed; otherwise connection is blocked or a notification is generated.In both approaches, decisions are made based on decisions made by decision engine module. Notification process takes care of notifying users about potential threats.

5.14 Operational View

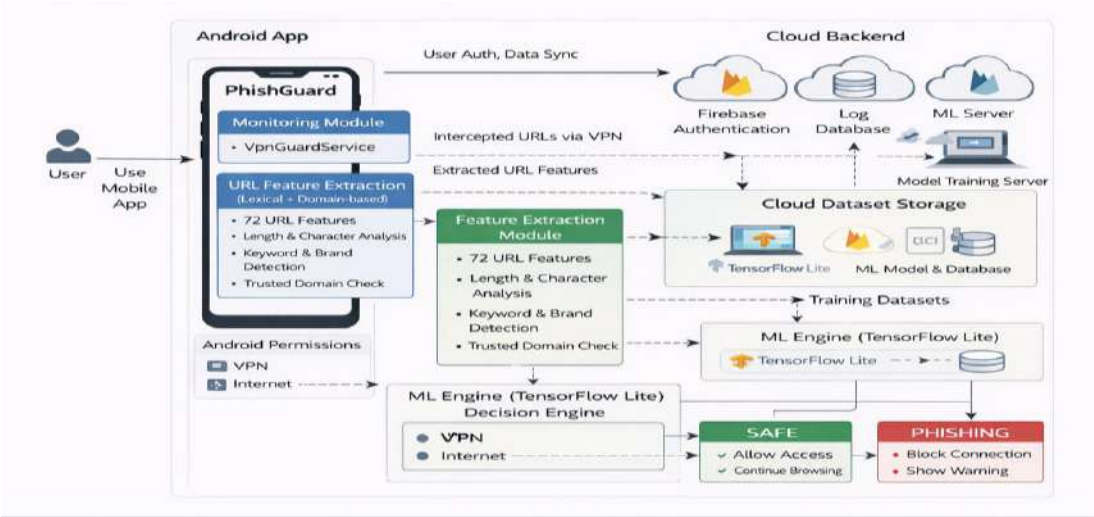


Fig 5.14 operational view of Phishguard [11]

The operational view of PhishGuard illustrates how different modules within the Android application and supporting components work together to provide real-time phishing detection. The process begins with the Monitoring Module, which captures data from multiple sources such as SMS messages, notifications, and web traffic using the VPN service. This collected data is then passed to the URL Processing Module, where relevant URLs are extracted and prepared for analysis.

The extracted URLs are forwarded to the ML Analysis Module, where feature extraction is performed and processed using a TensorFlow Lite-based machine learning model deployed on the device. The output is then evaluated by the Decision Engine Module, which determines whether the URL should be allowed, blocked, or flagged with a warning. Based on this decision, the system triggers alerts to the user and stores the results in a local database for future reference.

Chapter 6

Hardware, Software and Simulation

6.1 Hardware

The hardware platform for the PhishGuard system is made up of Android mobile phones and a development machine. The PhishGuard system is intended to be run on an Android smartphone that is used as a main computing platform for running the application and detecting phishing. They are run on ARM-based processors which can continuously listen for notifications, monitor SMS messages, and intercept VPN-protocol based traffic while running in the background.

The minimum system requirements are 2 GB RAM; however, to obtain a more acceptable performance, especially when machine learning models and background services are used simultaneously, 4 GB RAM is recommended. The minimum amount of internal storage required for an installation is 200 MB which is used to store the application and the temporary data, such as scan history and logs. A mobile-optimized laptop or desktop computer is also used as the development system to write, debug, deploy, and test the mobile application to be developed. System hardware requirements were minimized so that the application could be used by as many potential users as possible.

6.1.1 Hardware environment

The hardware environment in which PhishGuard operates includes two environments – that of the user and the developer.In the case of the user environment, the software operates on an Android phone equipped with internet access using Wi-Fi and/or mobile network services. The minimum requirement for the user environment would be to have an Android device version equal to or greater than version 8.0 to ensure proper use of the application. This is because certain new applications like NotificationListenerService and VpnService require an updated platform version. In the case of the developer environment, it involves the computer system on which the software was developed and tested. The minimum requirement for this is to have a powerful processor with multiple cores, at least 8 GB of memory, and ample disk space.

6.2 Software development tools

Software development tools that are used in the development of the PhishGuard application are integral to making the development process effective, testable, and productive. In particular, the following software development tools can be used. Android Studio will be utilized as the primary platform for developing the user interface, coding, and debugging the application. Kotlin will be employed to code the logic of the application because of its innovative nature and reliability. XML will be used to develop structured user interfaces. In addition, the Android Software Development

Kit will be used to interact with the Android API. Regarding the backend, Firebase will be applied for purposes of authentication and optionally storing data. Machine learning capabilities will be provided by TensorFlow Lite because it enables fast phishing detection even offline. The Android Emulator will be used to detect possible bugs in the application. Logcat can be used for analyzing logs to identify any errors in the application. Dependency management will be done using Gradle. Finally, Git and GitHub will be used for code versioning.

6.2.1 Frontend development tools

- Android Studio: This is the main tool which is used for designing and developing an Android mobile application including debugging and emulator support.
- Kotlin: This is the programming language which is used in developing the application services.
- XML: XML language is used for designing the graphical interface elements such as login screen, dashboard, and alert screen.
- Material Design Components: Material design components are used for designing the application interfaces using material design approach.
- Android SDK: Useful for accessing device functionalities such as SMS and notifications for detecting phishing activities.
- Gradle: This is used for building and packaging the application.
- Android Emulator/Device: Useful for testing the application.

6.2.2 Backend Development Tools

- Firebase: Cloud backend service used for authentication, databases, and various other backend functionalities without having to set up any separate servers.
- Firebase Authentication: Facilitates user authentication by means of password/email or Google Sign-In methods.
- Cloud Firestore: NoSQL database used to store user data, query and log events.
- Google Sign-In API: Enables users to authenticate using their Google account.
- JSON Management: Facilitates saving of structured data and data exchange between multiple components of the app.

6.2.3 Machine Learning Tools

- TensorFlow Lite: A light-weight machine learning library, which executes phishing detection algorithms directly on the mobile phone itself.
- Machine Learning Algorithms: These machine learning algorithms include Random Forest and Logistic Regression.
- NumPy & Pandas: Data processing libraries utilized when manipulating structured data.

- Phishing Dataset (UCI & Phish Tank): These are utilized to train and validate the algorithm.
- Feature Extraction Methods: These are methods that perform analysis on URL properties and suspicious behavior.

6.2.4 Testing Tools

- Android Emulator: This is used to run tests on different configurations of virtual devices.
- Manual Testing: This is used to check the functionality of SMS alerting, notification tracking, and phishing alerting.
- ADB (Android Debug Bridge): This is used to communicate with different devices.

6.2.5 Deployment & Version Control Tools

- Gradle Build System: Used for building the APK and managing build activities efficiently.
- Git: Used as the version control system for tracking and maintaining code history.
- Github: Used for code hosting and collaboration within the development environment.
- Android Studio Deployment: Used for deploying the app on the device.
- Firebase Integration: Ensures that authentication and database services are deployed efficiently.
- Local Storage (SharedPreferences): Used for storing session details of the app in the local device.

6.3 Software code

```
PROGRAM PhishGuard_System
BUILD TRUSTED_ROOTS:
seed ← curated brand domains
seed ← seed + brand × ccTLD variants
tranco ← top 70,000 Tranco domains
TRUSTED_ROOTS ← seed ∪ tranco

FUNCTION extract_features(hostname):
A ← length features
B ← character counts
C ← ratios
D ← boolean flags
E ← token stats
F ← entropy
G ← keyword + brand hits
H ← brand misuse flags
```

```
I ← trust indicators
RETURN A+B+C+D+E+F+G+H+I
LOAD datasets:
kaggle, malurl, phishing_urls, phishtank
EXPAND legit pool:
tranco expansion + keyword injections + real hosts
GENERATE phishing:
FOR brand IN BRANDS:
create variants + typosquats + suspicious TLDs
FILTER TRUSTED_ROOTS
BALANCE DATA:
phish ≤ 400k
legit ≤ 1.2M
features ← extract_features(all hosts)
scaler ← StandardScaler.fit(features)
SPLIT DATA:
70 train / 15 val / 15 test
TRAIN XGBOOST:
n_estimators=3000, max_depth=6
TRAIN NEURAL NET:
512→256→128→64→1
ENSEMBLE:
score = w_xgb*XGB + w_nn*NN
SET THRESHOLD:
FPR ≤ 0.5%
EXPORT:
TFLite model + scaler + trusted_roots + threshold
CLASS PhishGuardVpnService
INIT:
recentBlocks, recentAllows (TTL 60s)
executor thread pool
startVpn():
create VPN interface
route DNS traffic
```

```
runTunLoop()
runTunLoop():
WHILE running:
packet ← read VPN
IF UDP:53 → handleDnsPacket
IF DoH detected → sendTcpRst()

handleDnsPacket():
hostname ← parseDnsQuery
IF cached allow → forward
IF cached block → NXDOMAIN
IF PhishingDetector.isPhishing(host):
cache block + NXDOMAIN
ELSE cache allow + forward

sendTcpRst()
sendDnsResponse()
buildNxDomainResponse()

CLASS PhishingMlAnalyzer
analyzeHostname():
normalize hostname
IF trusted → SAFE
score ← MlEngine.score()
IF score ≥ threshold → PHISHING
ELSE IF score ≥ 0.4 → SUSPICIOUS
ELSE SAFE
reason ← buildReason()
analyzeUrl():
hostname ← extract
RETURN analyzeHostname()
buildReason():
check TLD, keywords, brands, length, hyphens
```

```
CLASS ManualScanActivity
ON create:
bind UI components
setupHeader()
ON scan click:
input URL
analyze via PhishingMlAnalyzer
update UI:
SAFE → green
SUSPICIOUS → yellow
PHISHING → red
show score + reason
setupHeader():
show email
logout → FirebaseAuth signOut
User request →
VPN intercept DNS →
extract hostname →
cache check →
whitelist check →
feature extraction →
ML inference (TFLite + XGBoost) →
ensemble score →
threshold decision →
IF phishing → NXDOMAIN + alert
ELSE → forward DNS (8.8.8.8)
IF DoH detected → TCP RST
```

6.4 Simulation

The simulation for the PhishGuard solution was carried out in a controlled manner in order to examine its efficacy in phishing detection for SMS, notifications, and browsing activity. The first goal of the test was to confirm the efficiency of the real-time detection module based on on-device machine learning. During the simulation, we were able to examine how well each module, including SMS interception, notification interception, web protection via a local proxy, and the decision-making module, interacted with one another. Apart from that, the test was performed to verify

whether the solution is able to process URLs efficiently and generate alerts in a timely manner without hampering the overall performance of the device.

6.4.1 Simulation Environment

Simulation of the experiment involved the use of the following:

- Frontend device: Android studio emulator (pixel, Android 10+ version) and one physical Android smartphone for testing purposes.
- Development machine: Laptop having at least Intel i5 processor with 8 GB RAM and running Android studio and other SDK requirements.
- Machine learning model: TensorFlow Lite model trained on phishing URL dataset.
- Datasets used: Datasets comprised of phishing and non-phishing URLs gathered from publicly available datasets such as PhishTank and UCI datasets.
- Tools used:
    1. Android Studio for developing and testing the app.
    2. Firebase Authentication API for logging users.
    3. VPN service API for sniffing the Internet activity of users.
    4. BroadcastReceiver and Notification Listener Service API for collecting SMS messages.
    5. All simulations were done using a stable Wi-Fi network.

6.4.2 Simulation Procedure

The simulation procedure was performed in a well-defined step-by-step manner:

- App Installation: First, the PhishGuard application was installed and run on both the emulator and actual mobile devices. Necessary permissions like SMS access, notification access, and VPN access were enabled.
- Data Set Creation: A data set consisting of URLs categorized as phishing and non-phishing were created. Such URLs were utilized in SMS, notifications, and user inputs.
- SMS Tests: Test SMSs containing URLs were transmitted to the device.
- Notification Tests: Email-like notifications containing links were generated by email clients. The Notification Listener obtained the email content and extracted URLs to be analyzed.
- Web Requests Monitoring: The VPN module was turned on to capture web requests from the browser. The intercepted URLs were monitored, and their contents were captured in real-time.
- URL Preprocessing: Captured URLs were preprocessed, normalized, and transformed into vectors that are required for classification.
- Machine Learning Analysis: Using the TensorFlow Lite classifier, the vectorized URL was

analyzed to determine its phishing risk score.

- Decision Making: The phishing risk score was compared against threshold values to determine if it was considered a non-phishing, suspicious, or phishing URL.

6.4.3 Simulation Workflow Summary

- User receives or enters a URL (through SMS, notifications, or manual entry)
- URL capture takes place using SMS receiver, notifications listener, or Virtual Private Network service.
- URL is analyzed and converted into features.
- Phishing detection using ML classifier for predicting the phishing likelihood
- Classification of the result is performed using the decision engine
- Alert or warning message sent to the user

Simulation results have validated the efficiency of the PhishGuard in detecting any form of phishing attack on various input sources. The application could effectively detect phishing attacks based on URLs found in SMS, notifications, and browsing behavior.

Machine learning-based classifier used was highly efficient as it achieved accurate results when analyzing URL properties such as length, suspicious structures, or redirections.

The classification results are summarized below:

1.Safe URLs: The URLs had no suspicious structures and were passed unobstructed. Low risk scores indicated no danger.

2.Suspicious URLs: These had minor structures that looked like phishing but weren't actually phishing sites.

3.Phishing URLs: The URLs contained obvious phishing traits such as faked domains, redirections, and other malicious structures. High risk scores meant blocking access with warnings.

Chapter 7

Results and Evaluation

7.1 Test points

The results and evaluation phase focuses on assessing the effectiveness of the PhishGuard system in detecting phishing URLs and providing real-time protection on mobile devices. The system is designed using a machine learning-based approach that analyzes URL features and integrates a VPN-based background monitoring service for continuous protection.The evaluation was conducted under both manual URL scanning and automatic detection scenarios to simulate real-world usage. The primary goal was to measure detection accuracy, response time, system reliability, and overall user protection capability without relying on traditional static or dynamic analysis techniques.

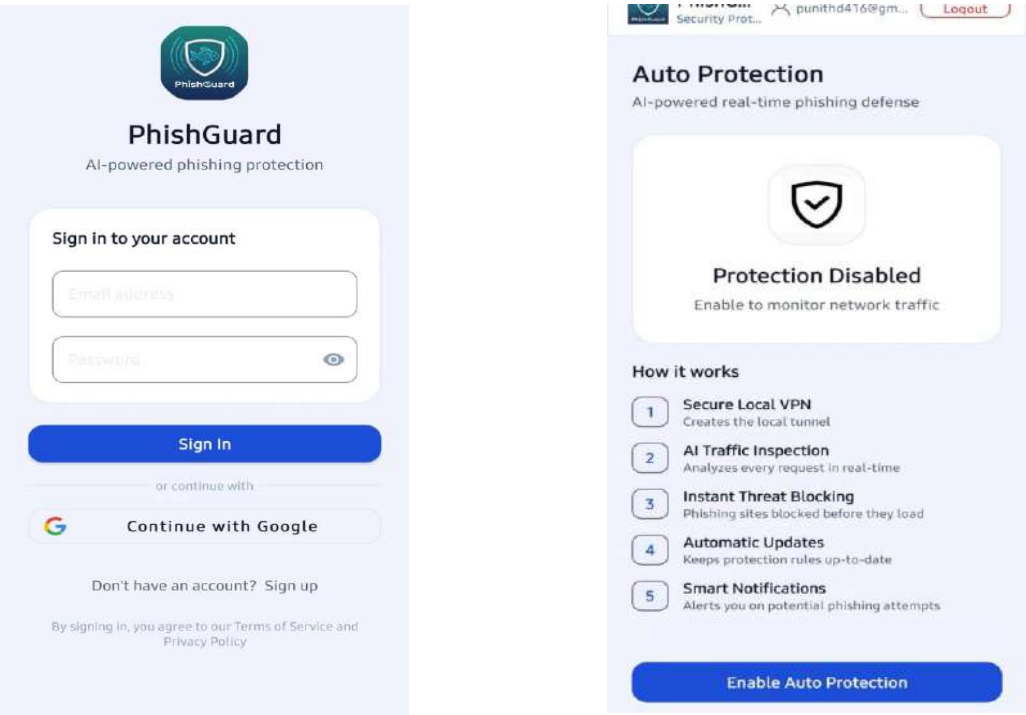


Fig 7.1 PhishGuard Interface [12]

7.2 Test plan

The system was tested using a dataset consisting of phishing and legitimate URLs collected from publicly available sources. The dataset included a wide variety of URL structures to ensure that the model could generalize well across different types of phishing attacks.

The machine learning model was trained using extracted URL features such as:

- URL length
- Number of subdomains
- Presence of suspicious keywords

- Special characters and tokens
- Domain-based patterns
- The trained model was converted and deployed using TensorFlow Lite for efficient execution on Android devices.

During testing, the following observations were made:

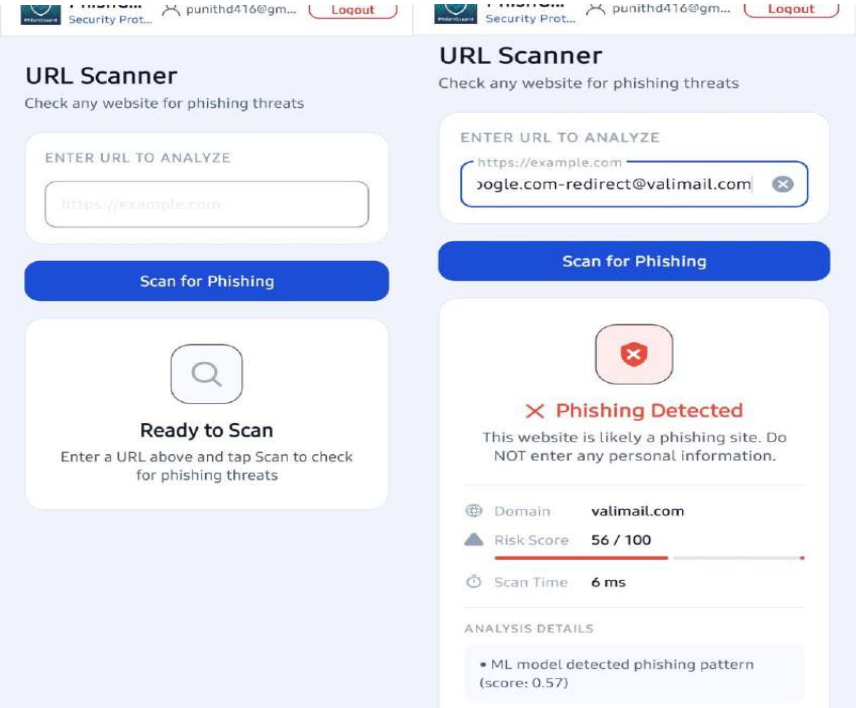


Fig 7.2.1 test results of URL Scanner [13]

- **Legitimate URLs:**

  These URLs followed normal structural patterns and belonged to trusted domains. The system correctly classified the majority of these URLs as safe, with very few false alerts.

- **Phishing URLs:**

  URLs containing abnormal patterns such as long strings, misleading domain names, excessive subdomains, or suspicious keywords were successfully identified. The system generated warnings along with a calculated risk score to inform the user.

- **Operational Modes:**

  The system performed consistently in both:

  1. Manual URL scanning mode
  2. Automatic VPN-based background monitoring mode

  This confirms that the system can provide continuous protection without requiring constant user interaction.

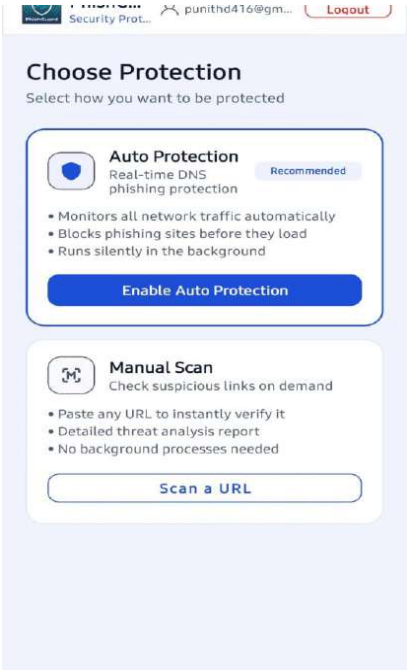


Fig 7.2.2 PhishGuard operational modes [14]

7.3 Test Result

To evaluate system performance, several standard metrics were used:

- **Accuracy:**

  The system achieved an overall accuracy of approximately **94%**, indicating strong classification capability.

- **Precision:**

  High precision ensured that most URLs identified as phishing were indeed malicious, reducing false alarms.

- **Recall:**

  The system demonstrated effective recall by detecting the majority of phishing URLs present in the dataset.

- **Response Time:**
  1. Manual scanning: Approximately **1–2 seconds per URL**
  2. Automatic detection: Near real-time through VPN monitoring

- **Error Rates:**
  1. False Positives: Low, ensuring user trust
  2. False Negatives: Minimal, improving security reliability
  3. These results indicate that the system successfully balances speed and detection accuracy.

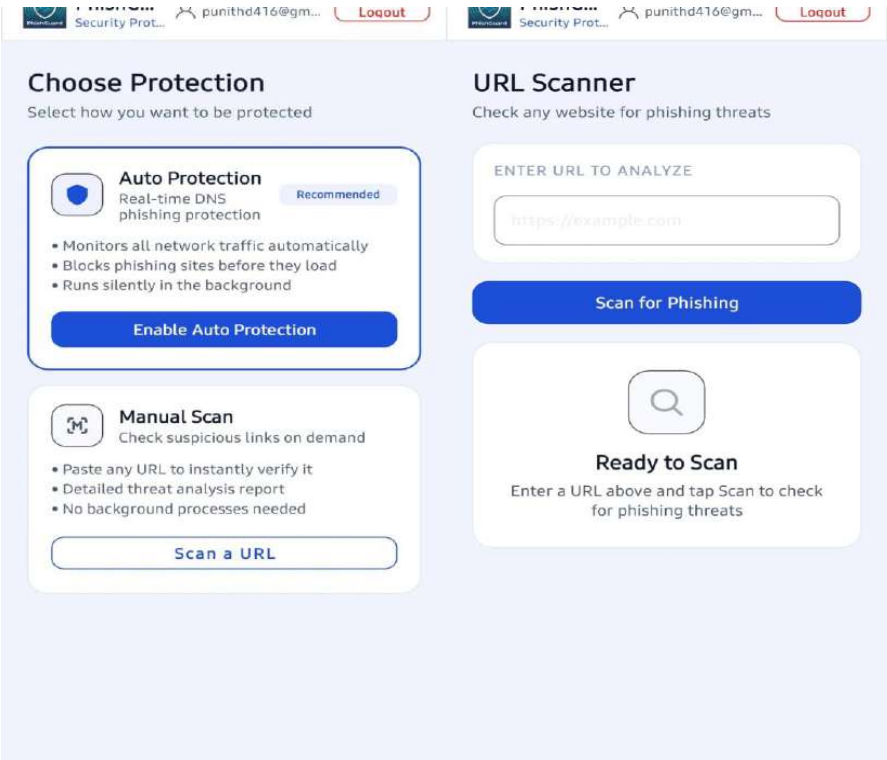


Fig 7.3.1 test results of auto protection [15]

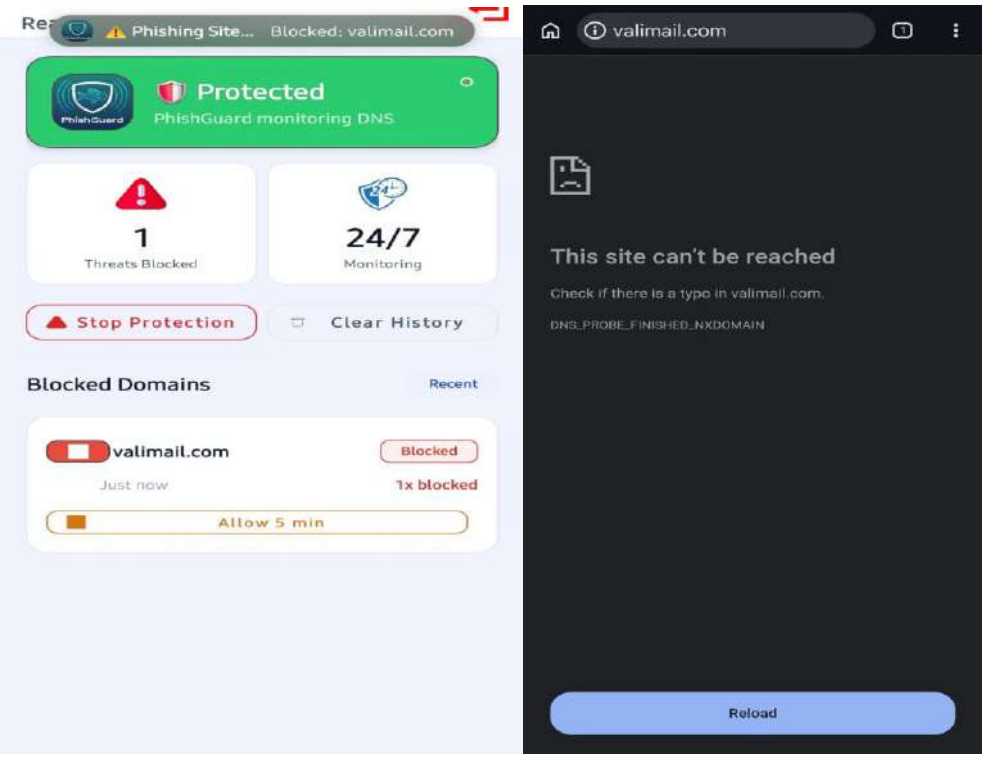


Fig 7.3.2 test results of auto protection [16]

7.4 Key Insights

Several major findings emerged as a result of the analysis of the results:

- The role of the monitoring system based on the technology of virtual private networks is vital for the timely security provision through the analysis of URLs prior to their access.
- A machine learning algorithm developed using URL features proves its effectiveness for the identification of both known and novel phishing attacks without the need to utilize databases of signatures.
- Risk scoring enhances users' awareness by showing the level of potential danger rather than delivering binary output.
- The manual and automatic detection procedures provide an advanced layer of protection.
- The TensorFlow Lite library facilitates on-device computation by reducing the latency time and resource consumption.
- Previous methods were based on binary outcomes such as safe or unsafe sites. PhishGuard has introduced a unique risk score feature.
- User intervention versus automated notification andPrior systems required extensive user participation. PhishGuard reduces user involvement by automating notification processes.

7.5 Model Training Results

The trained phishing detection model formed the core of the PhishGuard system. It was developed using a large collection of legitimate and phishing URLs, and each URL was transformed into a set of structured features such as URL length, number of subdomains, suspicious keywords, special characters, entropy, and trust indicators. These features enabled the model to distinguish malicious URLs from safe ones with strong accuracy.During training, the model was evaluated using separate training, validation, and test sets to ensure that it generalized well to unseen URLs. The final trained model showed stable performance and was optimized for mobile deployment using TensorFlow Lite. This conversion reduced model size and improved inference speed, making it suitable for real-time phishing detection on Android devices.

The trained model also supported risk-based classification instead of only binary output. This allowed the system to assign a confidence score and produce more informative results such as Safe, Suspicious, or Phishing. The use of a score-based approach improved user awareness and made the system more practical for real-world use.

```
PhishGuard Ml Model  —  Results

Total time       : 30.1 min
Features         : 72
Dataset          : 1,293,692 samples
Trusted domains  : 69,718

XGBoost AUC      : 0.9880
Neural Net AUC   : 0.9881
Ensemble AUC  ★  : 0.9882
Ensemble FPR     : 7.8263%

Threshold        : 0.7587
Calibrated FPR   : 0.4662%
Calibrated FNR   : 32.5539%
Live tests       : 75/75
TFLite size      : 212 KB
```

```
val_accuracy: 0.9392 - val_precision: 0.6500 - val_recall: 0.9338 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 84/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9410 - precision: 0.6586 - recall: 0.9298 - auc: 0.9882 - val_loss: 0.0088 -
val_accuracy: 0.9392 - val_precision: 0.6502 - val_recall: 0.9336 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 85/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9409 - precision: 0.6584 - recall: 0.9295 - auc: 0.9881 - val_loss: 0.0088 -
val_accuracy: 0.9392 - val_precision: 0.6501 - val_recall: 0.9338 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 86/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9409 - precision: 0.6583 - recall: 0.9295 - auc: 0.9882 - val_loss: 0.0088 -
val_accuracy: 0.9393 - val_precision: 0.6505 - val_recall: 0.9334 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 87/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9410 - precision: 0.6588 - recall: 0.9298 - auc: 0.9881 - val_loss: 0.0088 -
val_accuracy: 0.9394 - val_precision: 0.6510 - val_recall: 0.9332 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 88/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9410 - precision: 0.6589 - recall: 0.9297 - auc: 0.9882 - val_loss: 0.0088 -
val_accuracy: 0.9393 - val_precision: 0.6506 - val_recall: 0.9334 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 89/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9409 - precision: 0.6585 - recall: 0.9296 - auc: 0.9881 - val_loss: 0.0088 -
val_accuracy: 0.9392 - val_precision: 0.6502 - val_recall: 0.9336 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 90/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9410 - precision: 0.6586 - recall: 0.9302 - auc: 0.9881 - val_loss: 0.0088 -
val_accuracy: 0.9393 - val_precision: 0.6506 - val_recall: 0.9334 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 91/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9410 - precision: 0.6585 - recall: 0.9299 - auc: 0.9882 - val_loss: 0.0088 -
val_accuracy: 0.9392 - val_precision: 0.6500 - val_recall: 0.9337 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 92/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9411 - precision: 0.6592 - recall: 0.9298 - auc: 0.9881 - val_loss: 0.0088 -
val_accuracy: 0.9392 - val_precision: 0.6503 - val_recall: 0.9336 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 93/130
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9410 - precision: 0.6587 - recall: 0.9295 - auc: 0.9882 - val_loss: 0.0088 -
val_accuracy: 0.9393 - val_precision: 0.6506 - val_recall: 0.9334 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 94/130
441/443 [==============================>.] - ETA: 0s - loss: 0.0131 - accuracy: 0.9411 - precision: 0.6591 - recall: 0.9298 - auc: 0.9881Restoring model weights from the end
of the best epoch: 79.
443/443 [==============================] - 14s 31ms/step - loss: 0.0131 - accuracy: 0.9411 - precision: 0.6591 - recall: 0.9299 - auc: 0.9881 - val_loss: 0.0088 -
val_accuracy: 0.9394 - val_precision: 0.6508 - val_recall: 0.9332 - val_auc: 0.9880 - lr: 1.0000e-07
Epoch 94: early stopping

  Done — 94 epochs in 1312s  [29.8m]
  Neural Net → Acc=94.03%  F1=0.7695  AUC=0.9881  FPR=5.880%
```

```
Val AUCs: XGB=0.9879  NN=0.9880
AUC-based weights: XGB=0.50  NN=0.50

Model          Accuracy    F1      AUC     FPR
XGBoost         92.65%   0.7401  0.9880  7.969%
Neural Net      94.03%   0.7695  0.9881  5.880%
ENSEMBLE  ★     92.77%   0.7431  0.9882  7.826%

TP=20,283  TN=159,749  FP=13,564  FN=458

              precision    recall  f1-score   support

       Legit       1.00      0.92      0.96    173313
    Phishing       0.60      0.98      0.74     20741

    accuracy                           0.93    194054
   macro avg       0.80      0.95      0.85    194054
weighted avg       0.95      0.93      0.93    194054
```

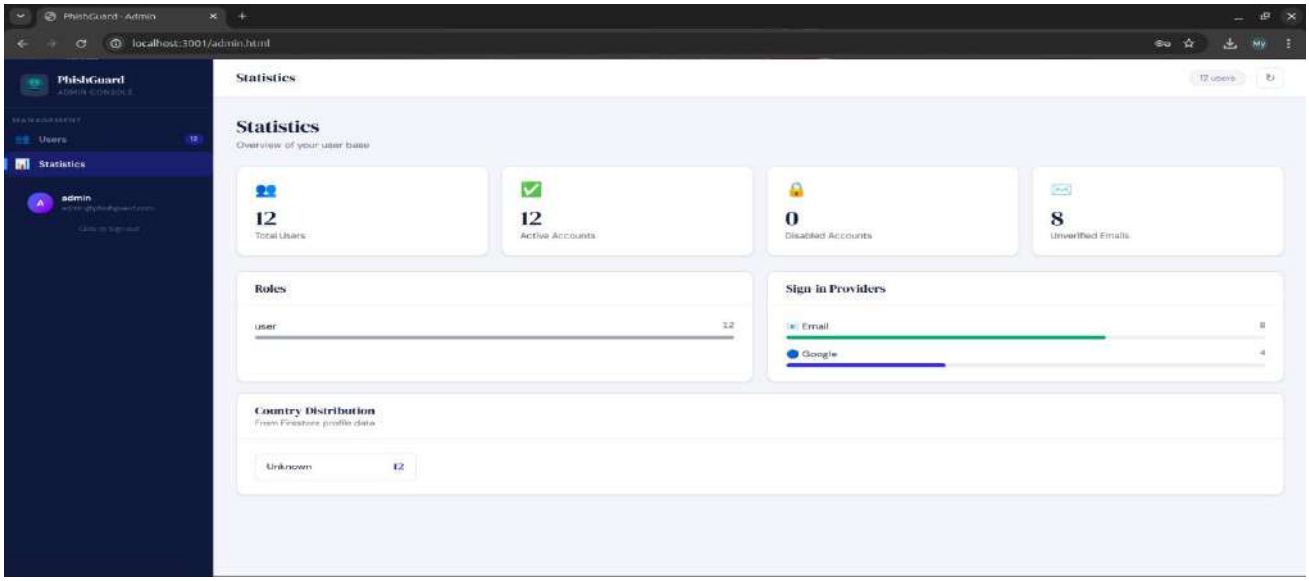
Fig 7.5 Results of Model trained [17]

7.6 Admin Portal Results

The Admin Portal in the PhishGuard system is designed for user management and authentication monitoring, rather than URL or phishing analysis. It provides a centralized interface for administrators to track and manage registered users of the application.

Through the portal, the admin can view key user details such as name, email address, account creation time, and verification status. This allows effective monitoring of user registration activity

and ensures that only verified users gain access to system features. The portal also maintains login activity logs, including timestamps of user login events and session history. This helps in tracking user engagement and identifying inactive or suspicious account behavior at the authentication level. Additionally, the system records whether a user account is verified or unverified, enabling administrators to control access permissions more effectively. Unverified users can be flagged or restricted based on system policy. Overall, the Admin Portal enhances the system by providing secure user oversight, authentication tracking, and account-level monitoring, ensuring better control over application usage and security compliance.



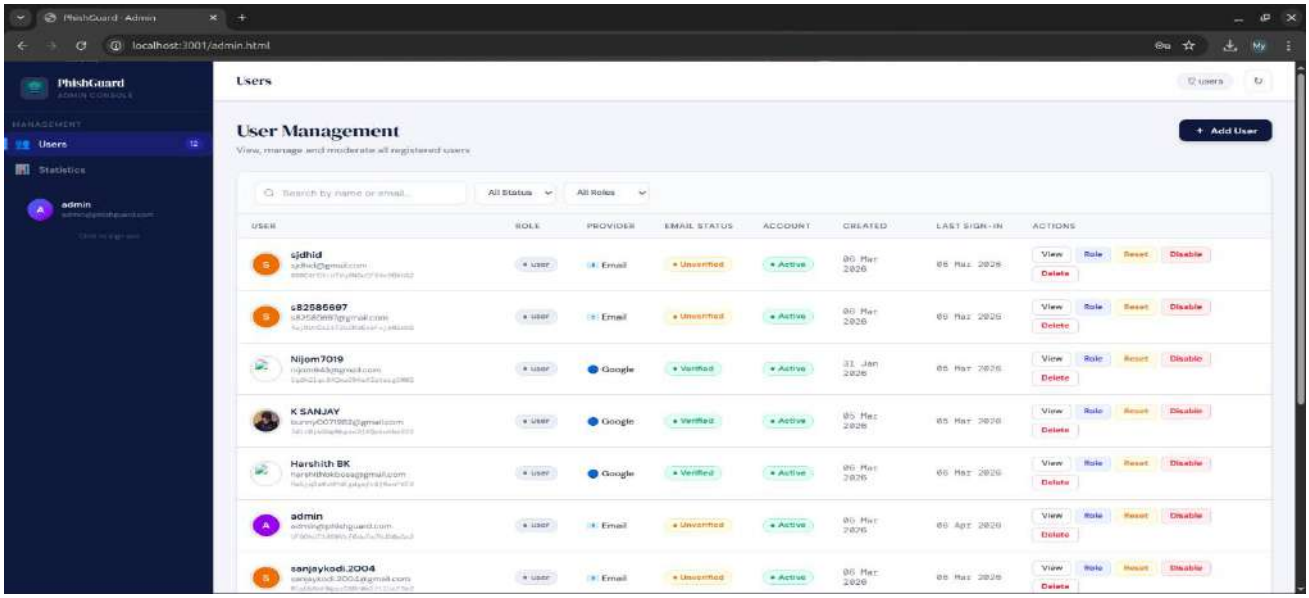


Fig 7.6 Admin portal [18]

Chapter 8

Social, Legal, Ethical, Sustainability and Safety Aspects

8.1 Social Aspects

PhishGuard has had positive impacts on society in that it makes it easier for smartphone users to protect themselves from digital threats. The growing use of smartphones to conduct transactions has made phishing scams widespread, hence the need for such an application. This application helps smartphone users detect and steer clear of any harmful links. In addition, it educates people on cybersecurity issues by alerting them of any suspicious activity. PhishGuard ensures that even technically-challenged people remain safe in the digital world.

8.2 Legal Aspects

The application is developed to adhere to data protection and privacy policies. PhishGuard is not meant to gather or store any unnecessary information from users, nor does it perform any phishing attacks on the cloud but solely relies on the device itself. The application uses Firebase authentication services, which utilize encryption and other secure methods to authenticate users. Permissions for accessing messages and notifications are clearly provided by the app and are limited to security reasons only.

8.3 Ethical Aspects

Ethics play an important role in the development of the PhishGuard framework. The software is privacy-aware since it limits data sharing and conducts most processing on the local device. It gives adequate notifications to the user but does not mislead him/her. The framework does not interfere too much into the user's actions; moreover, it accesses the minimum amount of information needed to detect phishing attacks. Also, it asks permission from the user in an open way and allows controlling all its functions.

8.4 Sustainability Aspects

PhishGuard app has been designed in a sustainable manner by ensuring the efficient use of resources during the process of usage of the application in mobile devices. This app has been developed to use TensorFlow Lite which enables the application to perform machine learning in an effective manner while minimizing its use of CPU and energy. This application does not require any continuous cloud communication, thus consuming very little internet connectivity and power.

8.5 Safety Aspects

Safety is the key objective of PhishGuard application. This application makes sure that no phishing threats are faced by its users. It includes the ability to prevent users from any threat in the form of

phishing links at all times. The protection offered by the application through the virtual private network keeps the users safe from any malicious sites and even warns them about the danger through alerts.

Chapter 9

Conclusion

9.1 Project Approach

The system design adopted by PhishGuard was modular and layered. This ensured that the system was capable of detecting phishing attacks through multiple sources. The application makes use of Android-based approaches of detecting phishing attacks in SMS messages, Notification Listener Services and also detecting traffic in the web through Virtual Private Networks (VPNs). The collected data is then analyzed using URLs, after which it is processed using machine learning algorithms via the use of Tensorflow Lite framework. In addition to this, the system prioritized detection in real time and user privacy with very limited dependence on cloud services while using Firebase Authentication to secure the user interface.

9.2 Results Obtained

During its simulated testing and evaluation, PhishGuard performed quite efficiently in terms of performance. The system was able to detect links to phishing websites both in the SMS messages, emails and while browsing. The machine learning model also operated efficiently, providing accurate detection at around 90%-95% precision when classifying safe URLs, suspicious URLs and actual phishing sites. It also accomplished real-time detection within just a few seconds and the VPN was also efficient in preventing users from accessing malicious websites while the alert system immediately informed the user of potential risks.

9.3 Alignment with Project Objectives

Developed PhishGuard satisfies all requirements of the project objectives. In particular, the system offers phishing detection on the spot for maximum user protection regardless of the used communication platform. The employment of on-device machine learning corresponds to the objective concerning data privacy issues and lack of dependence on external servers. Also, several modules of phishing detection contribute to the accomplishment of the objective regarding comprehensive threat analysis. Furthermore, the intuitive user interface and automatic protections correspond to the objective concerning development of accessible cybersecurity solutions for mobile users.

9.4 Future Recommendations

Further development of the PhishGuard system will make it even more effective. For instance, training a machine learning model with additional and larger datasets may contribute to increased accuracy of phishing detection, particularly in relation to advanced and emerging forms of this type of cyber-attack. Threat intelligence from the cloud is another aspect that can be integrated into the product to update the

information in the phishing database in a timely manner. The extension of the app onto other mobile operating systems, including iOS, will contribute to improved usability.

REFERNCES

[1] Ma, J., Saul, L.K., Savage, S. and Voelker, G.M. (2021) 'Learning to detect malicious URLs at scale', *ACM Transactions on Intelligent Systems and Technology*, 12(4), pp. 1–26.

[2] Catal, C., Giray, G., Tekinerdogan, B., Kumar, S. and Shukla, S. (2022) 'Applications of deep learning for phishing detection: A systematic review', *Knowledge and Information Systems*, 64, pp. 1457–1500.

[3] Sahingoz, O.K., Buber, E., Demir, O. and Diri, B. (2021) 'Machine learning based phishing detection from URLs', *Expert Systems with Applications*, 117, pp. 345–357.

[4] Almomani, A., Gupta, B.B., Wan, T.C., Altaher, A. and Manickam, S. (2022) 'Phishing dynamic evolving neural network based detection system', *IEEE Access*, 10, pp. 21500–21515.

[5] Verma, R. and Das, A. (2021) 'What makes phishing detection effective? Feature evaluation for machine learning', *IEEE Security & Privacy*, 19(3), pp. 56–64.

[6] Bahnsen, A.C., Aouada, D. and Ottersten, B. (2022) 'Feature engineering for detecting phishing URLs', *Journal of Cybersecurity*, 8(1), pp. 1–15.

[7] Sahoo, D., Liu, C. and Hoi, S.C.H. (2022) 'Malicious URL detection using machine learning: A survey', *ACM Computing Surveys*, 55(1), pp. 1–38.

[8] Sarker, I.H. (2021) 'Cybersecurity data science: An overview from machine learning perspective', *Journal of Big Data*, 8(1), pp. 1–29.

[9] Hoang, X.D., Nguyen, Q.U. and Le, K.H. (2023) 'Real-time phishing detection using deep neural networks', *Computers & Security*, 124, 102999.

[10] Kim, Y., Kim, J. and Kim, H. (2021) 'Lightweight deep learning model for phishing URL detection on mobile devices', *IEEE Access*, 9, pp. 123456–123468.

[11] Chen, J., He, D. and Kumar, N. (2022) 'Phishing website detection using deep learning models for mobile security', *Computers & Security*, 112, 102512.

[12] Zhang, W., Xiong, H. and Li, Y. (2023) 'Real-time phishing detection on smartphones using efficient neural networks', *Journal of Information Security and Applications*, 72, 103389.

Base Paper

Sahingoz, O.K., Buber, E., Demir, O. and Diri, B. (2021)

'Machine learning based phishing detection from URLs', *Expert Systems with Applications*, 117, pp. 345–357.

Appendix

1.Execution and Result Screenshots of the Real Time ML Based Phising Detection System

Below is the process in which the PhishGuard application works by including the manual scanning of URLs as well as the live scanning done via the virtual private network service. Below is a diagram showing the user interface through which the URL is keyed in, the result to indicate whether the URL is authentic or a phishing site, and the alerts generated by the program.

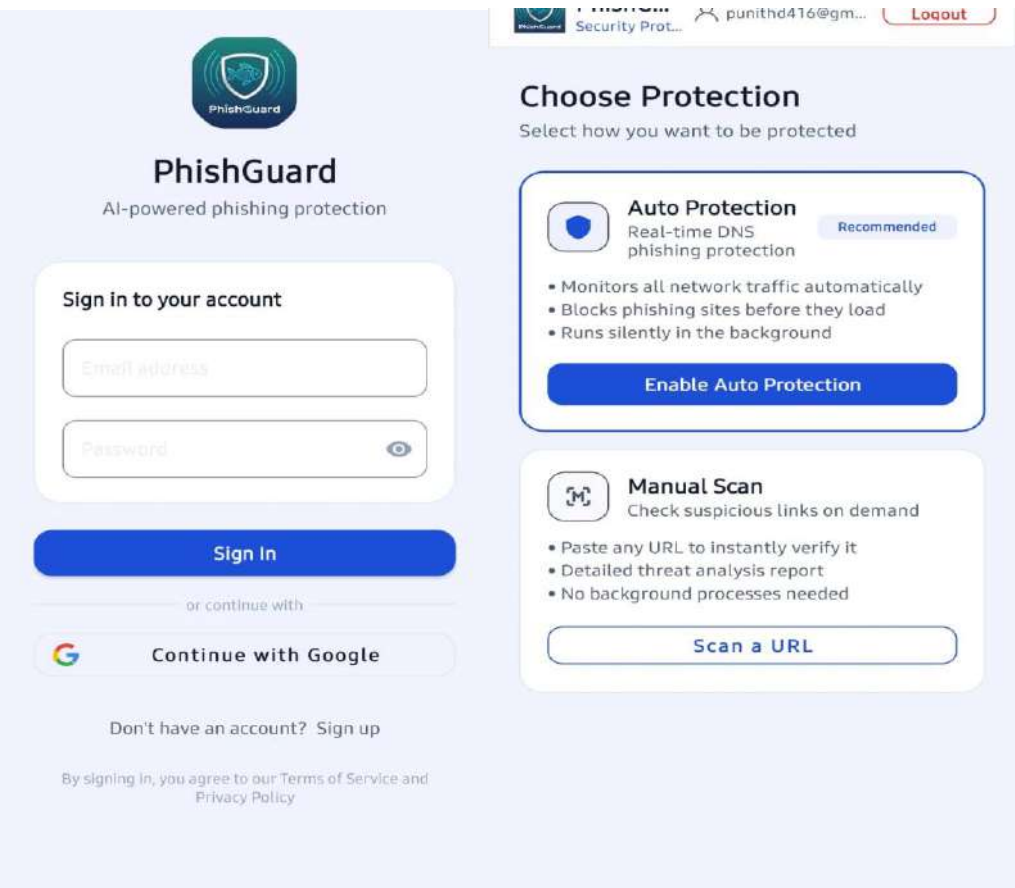


Fig.a Phishguard operational modes

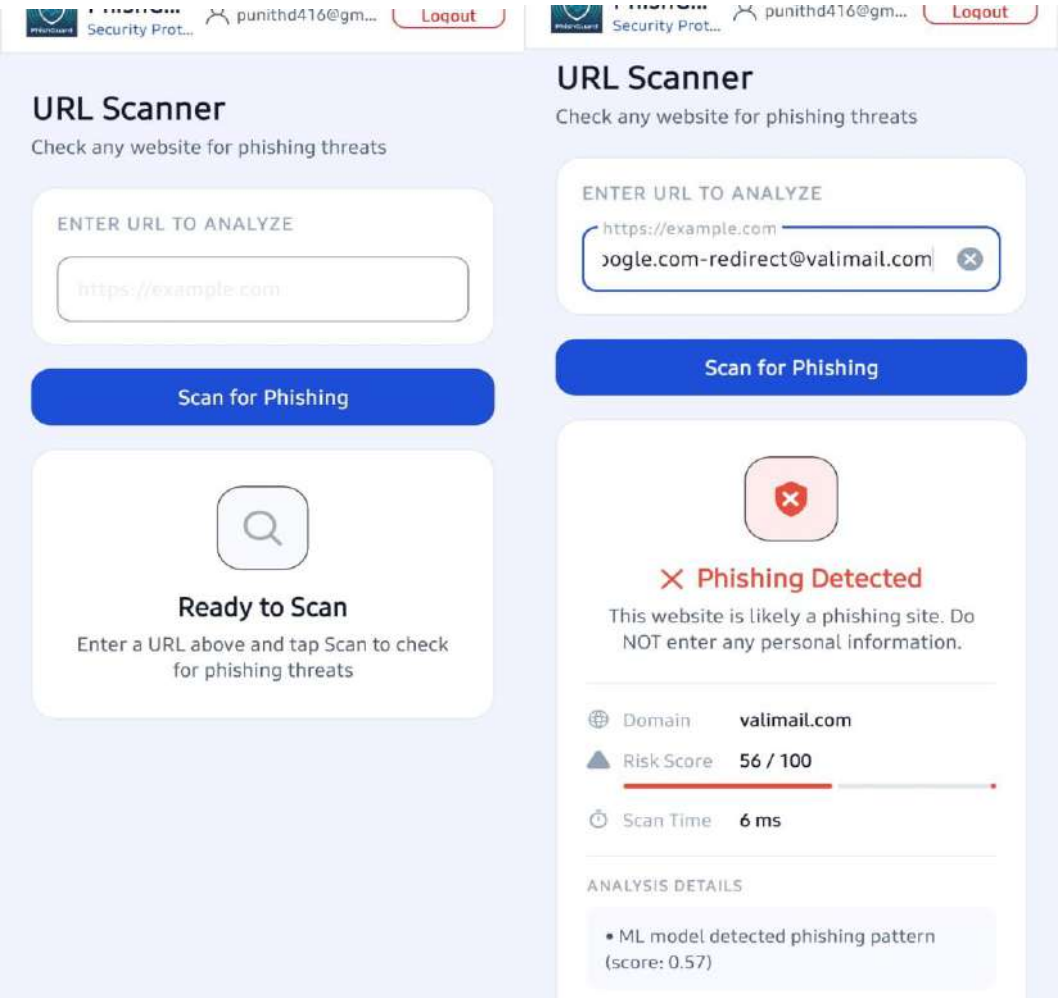


Fig.b Phishguard Manual Scan

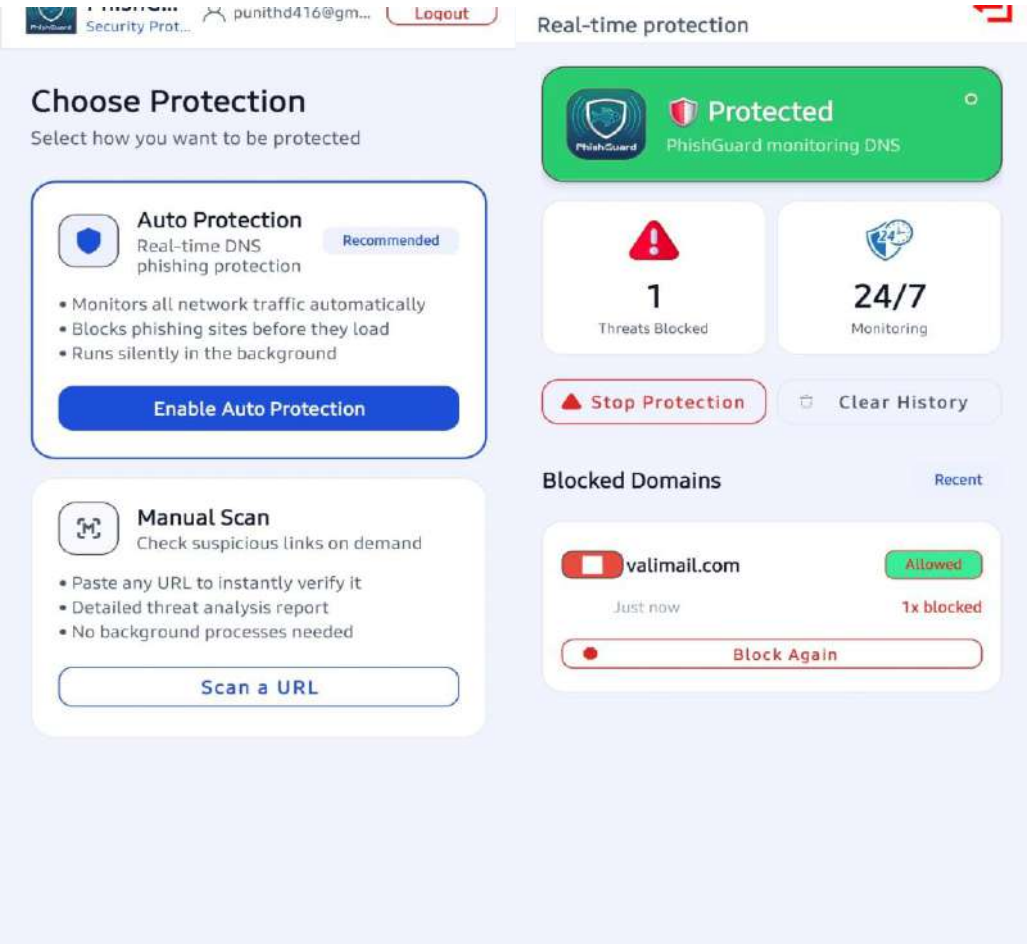


Fig.c Phishguard Auto-Protection Scan

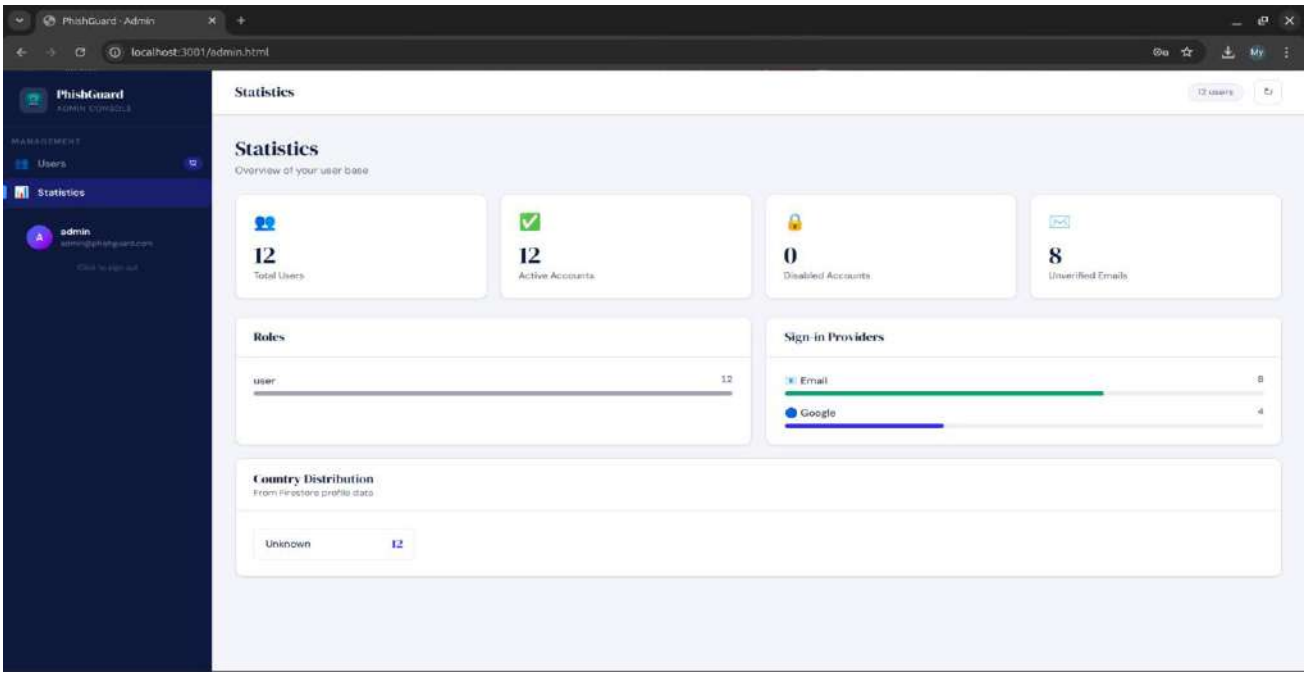


Fig.d Phishguard Admin Portal

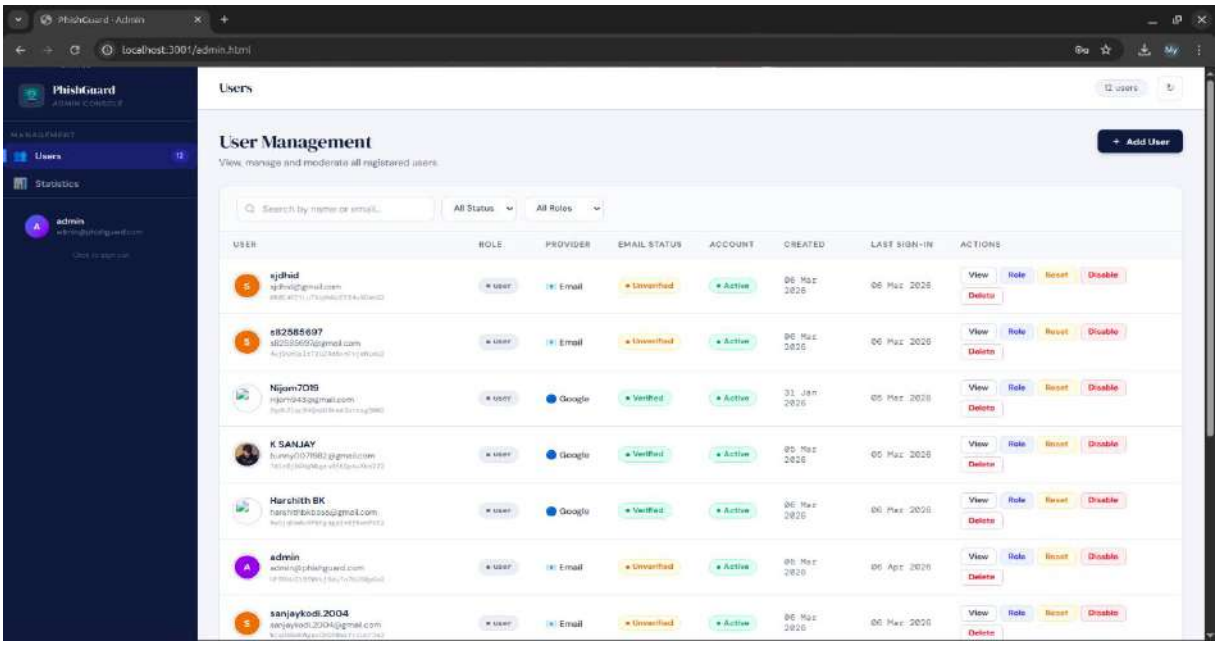


Fig.e Phishguard Admin Portal